

LAPORAN TUGAS KECIL 2
PEMBUATAN DOCKER CONTAINER
INSTITUT TEKNOLOGI BANDUNG

Mata Kuliah: Teknologi Platform (II2210)

Dosen pengajar : Dr. Ir. Albarda, M.T.



Disusun oleh:

Nama: Laras Hati Mahendra

Nim: 18223118

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2025

Daftar Isi

1. Instalasi Docker	4
1.1. Login Virtual Mesin	4
1.2. Update Paket	5
1.3. Instalasi Dependensi	8
1.4. Tambah Repository Docker	9
1.5. Instalasi Docker	11
1.6. Menambahkan User ke Group docker	12
2. Pembuatan Repository Menggunakan Git	14
2.1. Update dan Instalasi prerequisite	14
2.2. Menambah GPG key Docker	16
2.3. Menambah repo Docker	17
2.4. Update Ulang APT	18
2.5. Instalasi Docker	19
2.6. Memastikan docker berjalan	21
2.7. Menjalankan contoh container	22
2.8. Me-run "Hello World"	24
2.9. Instalasi GIT di VM	25
2.10. Membuat Repository Baru di GitHub	26
2.11. Menambahkan Deploy Key ke Repository	28
2.12. Menambahkan Deploy Key ke Repository	31
3. Pembuatan Web Server Sederhana (FastAPI)	36
3.1. Menginstalasi pip	36
3.2. Melakukan Cloning Repo	39
3.3. Melengkapi struktur file	40
3.4. Melengkapi main.py	40
3.4.1. Endpoint: GET/	40
3.4.2. Endpoint: GET/motd	41
3.4.3. Endpoint: POST/motd	43
3.4.4. Autentikasi	46
3.4.5. Kode backend Python pakai FastAPI	47
3.5. Membuat Dockerfile (Agar web Server Bisa Dijalankan di Port 17787)	49
Perintah yang digunakan:	49
3.6. Membuat index.html	51
3.7. Membuat model.py	59
3.8. Membuat requirements.txt	61
3.9. Membuat tester.py	62
3.10. Membuat docker-compose	65
3.11. Menjalankan Web Server menggunakan server menggunakan Docker Composer	67

3.12. Konfigurasi akses port dan Uji Endpoint /motd (GET)	68
3.13. Pengujian Endpoint /motd (GET)	70
3.14. Pengujian Endpoint /motd (POST)	71
4. Deployment dan Publikasi	72
4.1. Publikasi Kode ke GitHub (git remote)	72
4.2. Autentikasi SSH dan Cloning Repository	73
4.3. Penggunaan git add, commit, dan push	75
4.4. Deployment dengan Docker Compose	79
4.5. Mengedit dan Mengecek File docker-compose.yml	80
4.6. Menjalankan Docker Compose	81
4.7. Mengecek Container yang Sedang berjalan	82
4.8. Menampilkan Tampilkan Repo GitHub	82
5. Instalasi Docker dan Git di Virtual Machine	83
5.1. Menjalankan Container	83
5.2. Pengujian Endpoint //motd (GET)	84
5.3. Pengujian Endpoint /motd (POST)	85
6. Bonus	88
6.1. Tampilan Endpoint GET /motd (Max +1)	88
6.2. Otentikasi pada Endpoint POST /motd (Max +0,1)	94
7. Catatan Tambahan (Disclaimer)	96
8. Referensi	96
9. Link Lainnya Mengenai Tugas	97

1. Instalasi Docker

1.1. Login Virtual Mesin

Perintah yang digunakan:

```
ssh -i "C:\Users\laras\.ssh\larasheart.pem" larashtm@20.2.164.44  
//langkah untuk masuk ke VM
```

Langkah-langkah dan Penjelasan:

Ip public sebelumnya 20.2.112.84

- Namun, karena untuk menghemat supaya tidak tersedot maka saya mengikuti arahan asisten yang menyarankan saya untuk menjeda semenetara/shut down, Sehingga ipnya berubah dari yang awalnya 20.2.112.84 menjadi 20.2.164.44.
- Maka untuk cara untuk masuknya yaitu:

```
ssh -i "C:\Users\laras\.ssh\larasVM.pem" larashtm@20.2.164.44 //langkah untuk  
masuk ke VM . Jangan lupa untuk mengaktifkan mesin nya dahulu.
```

Screenshot:

```
larashtm@larasVM: ~  
Microsoft Windows [Version 10.0.26100.3476]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\laras\.ssh>ssh -i "C:\Users\laras\.ssh\larasheart.pem" larashtm@20.2.164.44  
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1025-azure x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:       https://ubuntu.com/pro  
  
System information as of Mon Apr  7 00:48:46 UTC 2025  
  
System load:  0.02          Processes:            110  
Usage of /:   6.0% of 61.84GB Users logged in:        0  
Memory usage: 44%          IPv4 address for eth0: 10.1.1.4  
Swap usage:   0%  
  
* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s  
  just raised the bar for easy, resilient and secure K8s cluster deployment.  
  
  https://ubuntu.com/engage/secure-kubernetes-at-the-edge  
  
Expanded Security Maintenance for Applications is not enabled.  
  
0 updates can be applied immediately.  
  
Enable ESM Apps to receive additional future security updates.  
See https://ubuntu.com/esm or run: sudo pro status  
  
New release '24.04.2 LTS' available.  
Run 'do-release-upgrade' to upgrade to it.  
  
Last login: Sun Apr  6 15:48:06 2025 from 103.147.9.195  
larashtm@larasVM:~$
```

1.2. Update Paket

Perintah yang digunakan:

```
sudo apt update && sudo apt upgrade -y
```

Langkah-langkah dan Penjelasan:

Perintah **sudo apt update && sudo apt upgrade -y** dipakai untuk memperbarui sistem baik paket yang tersedia supaya tahu versi terbaru dari software software yang tersedia di vm larasVm. Lalu akan muncul:

1. **Pending kernel upgrade:**

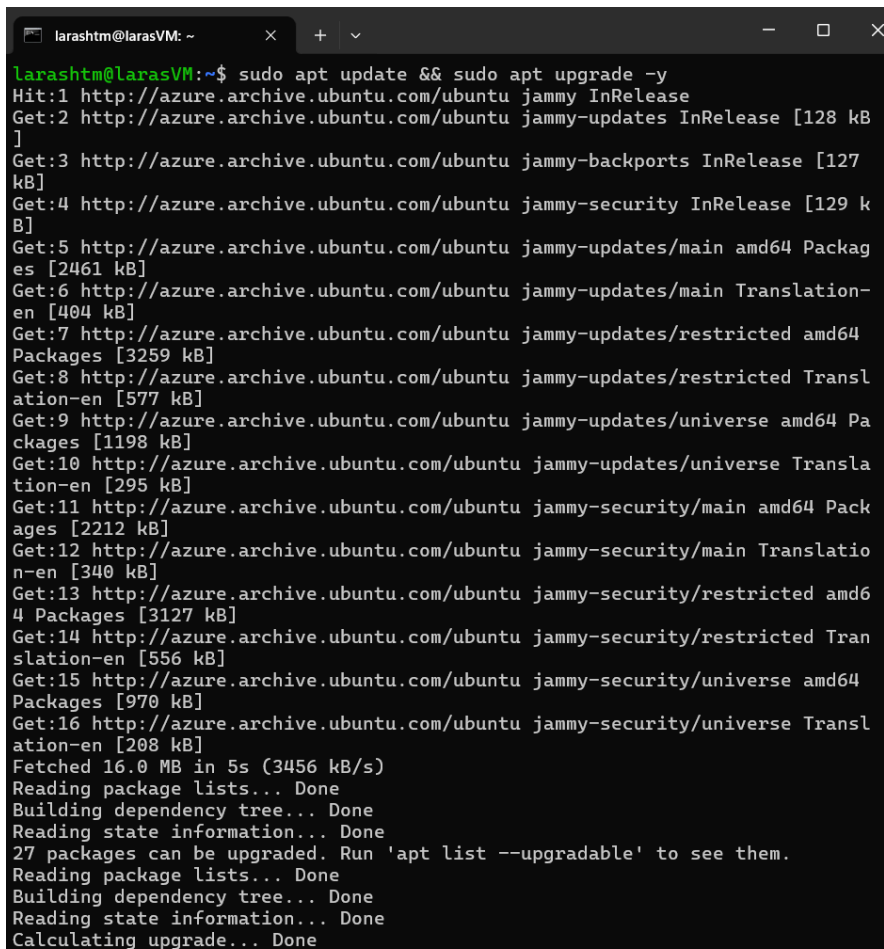
Sistem nya udah download versi kernel (inti sistem Linux) yang lebih baru, tapi kernel itu belum aktif karena belum restart. Kernel baru ini cuma akan mulai dipakai setelah sistem di-reboot. Jadi, walaupun udah di-upgrade, belum

bener-bener “make” kernel baru itu sampai restart.

2. **Daemons using outdated libraries:**

Daemons itu program atau service yang jalan di background. Pesan ini muncul karena ada daemon yang masih jalan pakai library versi lama, padahal library-nya udah di-update. Daemon-daemon ini perlu di restart atau sistemnya di root supaya mereka bisa pakai versi library yang baru.

Screenshot:



```
larashtm@larasVM: ~  
larashtm@larasVM:~$ sudo apt update && sudo apt upgrade -y  
Hit:1 http://azure.archive.ubuntu.com/ubuntu jammy InRelease  
Get:2 http://azure.archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]  
Get:3 http://azure.archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]  
Get:4 http://azure.archive.ubuntu.com/ubuntu jammy-security InRelease [129 kB]  
Get:5 http://azure.archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [2461 kB]  
Get:6 http://azure.archive.ubuntu.com/ubuntu jammy-updates/main Translation-en [404 kB]  
Get:7 http://azure.archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages [3259 kB]  
Get:8 http://azure.archive.ubuntu.com/ubuntu jammy-updates/restricted Translation-en [577 kB]  
Get:9 http://azure.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1198 kB]  
Get:10 http://azure.archive.ubuntu.com/ubuntu jammy-updates/universe Translation-en [295 kB]  
Get:11 http://azure.archive.ubuntu.com/ubuntu jammy-security/main amd64 Packages [2212 kB]  
Get:12 http://azure.archive.ubuntu.com/ubuntu jammy-security/main Translation-en [340 kB]  
Get:13 http://azure.archive.ubuntu.com/ubuntu jammy-security/restricted amd64 Packages [3127 kB]  
Get:14 http://azure.archive.ubuntu.com/ubuntu jammy-security/restricted Translation-en [556 kB]  
Get:15 http://azure.archive.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [970 kB]  
Get:16 http://azure.archive.ubuntu.com/ubuntu jammy-security/universe Translation-en [208 kB]  
Fetched 16.0 MB in 5s (3456 kB/s)  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
27 packages can be upgraded. Run 'apt list --upgradable' to see them.  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
Calculating upgrade... Done
```

```

larashtm@larasVM: ~
Processing triggers for dbus (1.12.20-2ubuntu4.1) ...
Processing triggers for install-info (6.8-4build1) ...
Processing triggers for linux-image-6.8.0-1025-azure (6.8.0-1025.30~22.04.1)
...
/etc/kernel/postinst.d/initramfs-tools:
update-initramfs: Generating /boot/initrd.img-6.8.0-1025-azure
/etc/kernel/postinst.d/zz-update-grub:
Sourcing file '/etc/default/grub'
Sourcing file '/etc/default/grub.d/40-force-partuuid.cfg'
Sourcing file '/etc/default/grub.d/50-cloudimg-settings.cfg'
Sourcing file '/etc/default/grub.d/init-select.cfg'
Generating grub configuration file ...
GRUB_FORCE_PARTUUID is set, will attempt initrdless boot
Found linux image: /boot/vmlinuz-6.8.0-1025-azure
Found initrd image: /boot/initrd.img-6.8.0-1025-azure
Found linux image: /boot/vmlinuz-6.8.0-1021-azure
Found initrd image: /boot/initrd.img-6.8.0-1021-azure
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
Adding boot menu entry for UEFI Firmware Settings ...
done
Scanning processes...
Scanning candidates...
Scanning linux images...

Restarting services...
/etc/needrestart/restart.d/dbus.service
systemctl restart multipathd.service networkd-dispatcher.service packagekit
.service polkit.service rsyslog.service ssh.service walinuxagent.service
Service restarts being deferred:
systemctl restart systemd-logind.service
systemctl restart unattended-upgrades.service
systemctl restart user@1000.service

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
larashtm@larasVM:~$

```

Package configuration

Pending kernel upgrade

Newer kernel available

The currently running kernel version is 6.8.0-1021-azure which is not the expected kernel version 6.8.0-1025-azure.

Restarting the system to load the new kernel will not be handled automatically, so you should consider rebooting.

<Ok>



1.3. Instalasi Dependensi

Perintah yang digunakan:

```
sudo apt install -y ca-certificates curl gnupg
```

Langkah-langkah dan Penjelasan:

Perintah **sudo apt install -y ca-certificates curl gnupg** dipakai untuk mendownload tools yang dibutuhkan dalam pengerjaan tugas saat ingin menambah respo atau install lainnya. Tiga aspek yang harus di download :

1. **ca-certificates**: supaya sistem mengenal dan menyetujui sertifikasi keamanan dari HTTPS.
2. **curl**: mengambil data dari internet lewat terminal, misalnya untuk download file atau mengakses.
3. **gnupg** : enkripsi dan verifikasi tanda tangan digital, biasanya dipakai untuk memastikan file yang kita download itu aman dan resmi. Flag **-y** dipakai biar semuanya langsung ke install otomatis tanpa harus konfirmasi satu-satu.

Screenshot:


```
larashtm@larasVM: ~  
larashtm@larasVM:~$ sudo apt install -y ca-certificates curl gnupg  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
ca-certificates is already the newest version (20240203~22.04.1).  
ca-certificates set to manually installed.  
curl is already the newest version (7.81.0-1ubuntu1.20).  
curl set to manually installed.  
gnupg is already the newest version (2.2.27-3ubuntu2.3).  
gnupg set to manually installed.  
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.  
larashtm@larasVM:~$
```

1.4. Tambah Repository Docker

Perintah yang digunakan:

```
sudo install -m 0755 -d /etc/apt/keyrings  
  
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo tee  
/etc/apt/keyrings/docker.gpg > /dev/null  
  
sudo chmod a+r /etc/apt/keyrings/docker.gpg  
  
echo \  
  
"deb [arch=$(dpkg --print-architecture)  
signed-by=/etc/apt/keyrings/docker.gpg]  
https://download.docker.com/linux/ubuntu \  
  
$(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list  
> /dev/null  
  
sudo apt update
```

Langkah-langkah dan Penjelasan:

1. **sudo install -m 0755 -d /etc/apt/keyrings** akan membuat folder

bernama **keyrings** di dalam **/etc/apt**, yang nantinya dipakai untuk menyimpan file GPG key dengan permission yang aman. Setelah itu, **curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo tee /etc/apt/keyrings/docker.gpg > /dev/null** akan mengambil GPG key milik Docker dari internet, terus menyimpan hasilnya ke dalam file **docker.gpg** di folder keyrings tadi. Ini penting agar sistem bisa verifikasi bahwa paket Docker yang nanti di-download itu benar-benar dari sumber resminya.

2. Selanjutnya, **sudo chmod a+r /etc/apt/keyrings/docker.gpg** memberikan akses baca ke semua user, supaya file GPG itu bisa dibaca oleh sistem APT tanpa masalah.
3. Lalu, baris **/etc/apt/keyrings/docker.gpg > /dev/null** akan menambahkan alamat repository Docker ke daftar sumber APT. Bagian **echo \"deb [arch=\$(dpkg --print-architecture) akan menyesuaikan arsitektur sistem (misal: amd64), dan \$(lsb_release -cs) akan otomatis mengambil codename dari Ubuntu yang dipakai, jadi repo-nya cocok sama sistem.**
4. Terakhir, **sudo apt update** dipakai untuk refresh daftar paket, supaya sistem ini tahu bahwa sekarang ada repo Docker yang sudah bisa dipakai untuk install Docker.

Screenshot:

```
larashtm@larasVM: ~$ sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo tee /etc/apt/
keyrings/docker.gpg > /dev/null
sudo chmod a+r /etc/apt/keyrings/docker.gpg
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker
.gpg] https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list
> /dev/null
sudo apt update
Hit:1 http://azure.archive.ubuntu.com/ubuntu jammy InRelease
Hit:2 http://azure.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:3 http://azure.archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:4 http://azure.archive.ubuntu.com/ubuntu jammy-security InRelease
Get:5 https://download.docker.com/linux/ubuntu jammy InRelease [48.8 kB]
Err:5 https://download.docker.com/linux/ubuntu jammy InRelease
  The following signatures couldn't be verified because the public key is no
t available: NO_PUBKEY 7EA0A9C3F273FCD8
Reading package lists... Done
W: GPG error: https://download.docker.com/linux/ubuntu jammy InRelease: The
following signatures couldn't be verified because the public key is not avai
lable: NO_PUBKEY 7EA0A9C3F273FCD8
E: The repository 'https://download.docker.com/linux/ubuntu jammy InRelease'
  is not signed.
N: Updating from such a repository can't be done securely, and is therefore
disabled by default.
N: See apt-secure(8) manpage for repository creation and user configuration
details.
```

1.5. Instalasi Docker

Perintah yang digunakan:

```
git clone https://github.com/fastfetch-cli/fastfetch.git
```

```
sudo apt install -y docker-ce docker-ce-cli containerd.io
docker-buildx-plugin docker-compose-plugin
```

Langkah-langkah dan Penjelasan:

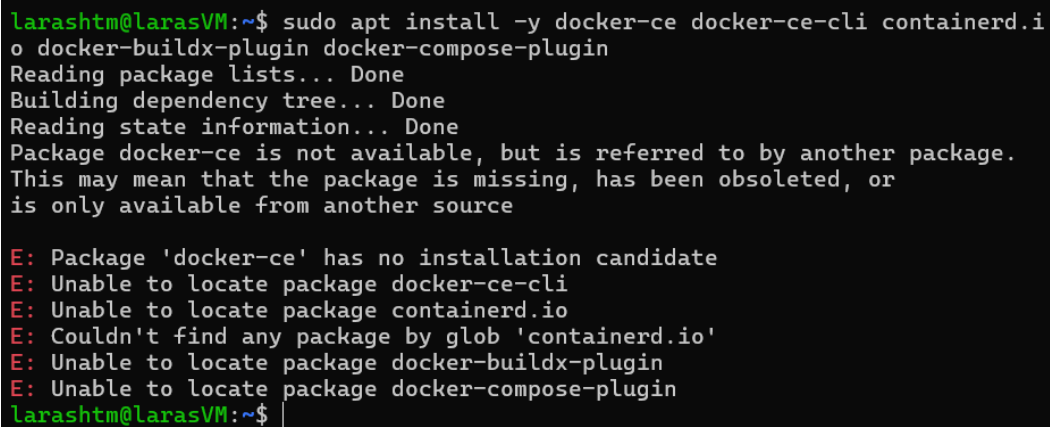
1. **git clone** <https://github.com/fastfetch-cli/fastfetch.git> digunakan untuk menginstal isi project Fastfetch dari GitHub ke komputer. Pada perintah ini akan membuat salinan (clone) dari repository bernama fastfetch yang ada di GitHub, dan menyimpannya di folder lokal dengan nama fastfetch. Di dalamnya

nanti ada semua file, folder, dan histori versi dari project itu.

2. **sudo apt install -y docker-ce docker-ce-cli containerd.io**

docker-buildx-plugin docker-compose-plugin artinya menginstal semua komponen penting Docker lewat perintah ini. Paket **docker-ce** adalah Docker Engine Community Edition, yaitu inti dari Docker-nya sendiri. Lalu **docker-ce-cli** adalah command-line interface-nya, agar bisa berjalan dengan perintah Docker lewat terminal. **containerd.io**, yang merupakan container runtime — ini kayak "mesin" di belakang layar yang menjalankan container. Selanjutnya, **docker-buildx-plugin** ditambahkan untuk membangun image Docker dengan lebih fleksibel, termasuk support untuk platform berbeda (cross-platform build). Terakhir, **docker-compose-plugin** memungkinkan menggunakan **docker compose** langsung, untuk mengatur dan menjalankan beberapa container sekaligus lewat satu file konfigurasi **docker-compose.yml**.

Screenshot:



```
larashtm@larasVM: ~$ sudo apt install -y docker-ce docker-ce-cli containerd.io
o docker-buildx-plugin docker-compose-plugin
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Package docker-ce is not available, but is referred to by another package.
This may mean that the package is missing, has been obsoleted, or
is only available from another source

E: Package 'docker-ce' has no installation candidate
E: Unable to locate package docker-ce-cli
E: Unable to locate package containerd.io
E: Couldn't find any package by glob 'containerd.io'
E: Unable to locate package docker-buildx-plugin
E: Unable to locate package docker-compose-plugin
larashtm@larasVM:~$
```

1.6. Menambahkan User ke Group docker

sudo usermod -aG docker sister

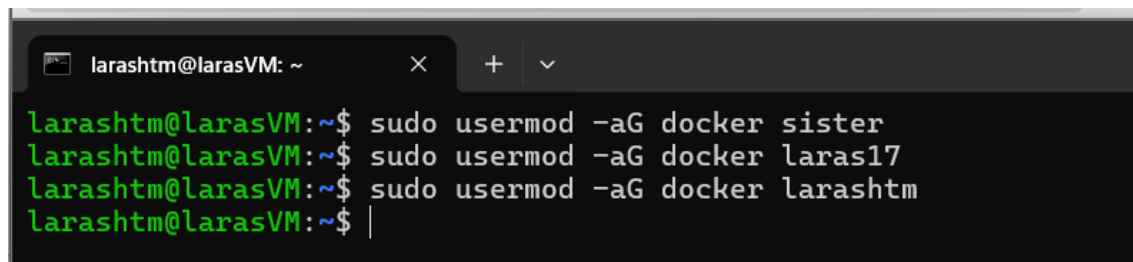
sudo usermod -aG docker laras17

```
sudo usermod -aG docker larashtm
```

Langkah-langkah dan Penjelasan:

Ketiga perintah itu dipakai untuk menambahkan user ke dalam grup **`docker`**, supaya mereka bisa menjalankan perintah **Docker** tanpa perlu pakai **`sudo`** lagi. User **`sister`**, **`laras17`**, dan **`larashtm`** masing-masing ditambahkan ke grup **`docker`** dengan opsi **`-aG`** yang artinya **“append to group”** tanpa menghapus grup lain yang udah ada. Setelah ini, user terkait harus logout dan login lagi supaya perubahan grupnya aktif.

Screenshot:



```
larashtm@larasVM: ~  
larashtm@larasVM:~$ sudo usermod -aG docker sister  
larashtm@larasVM:~$ sudo usermod -aG docker laras17  
larashtm@larasVM:~$ sudo usermod -aG docker larashtm  
larashtm@larasVM:~$ |
```

Catatan:

1. Apabila ingin clone namun tidak mau mengganggu folder lama, bisa memberi nama: **git clone https://github.com/fastfetch-cli/fastfetch.git fastfetch-baru**

```
larashtm@larasVM: ~$ git clone https://github.com/fastfetch-cli/fastfetch.git
fastfetch-baru
Cloning into 'fastfetch-baru'...
remote: Enumerating objects: 46505, done.
remote: Counting objects: 100% (1656/1656), done.
remote: Compressing objects: 100% (337/337), done.
remote: Total 46505 (delta 1473), reused 1347 (delta 1319), pack-reused 4484
9 (from 5)
Receiving objects: 100% (46505/46505), 12.46 MiB | 14.85 MiB/s, done.
Resolving deltas: 100% (32743/32743), done.
larashtm@larasVM:~$
```

2. Kalau di teman teman muncul Error itu karena **APT belum kenal sama paket docker-ce dan teman-temannya**, Secara default, Ubuntu nggak punya akses ke repo Docker, jadi langsung jalankan perintah install tanpa setup repo-nya dulu, APT bakal bilang kalau paketnya “nggak ditemukan.” Untuk itu, sebelum install Docker, penting untuk setup repo dahulu.

2. Pembuatan Repository Menggunakan Git

Nomor	Sifat	Keterangan
2.1 - 2.8	Tambahan	Persiapan Awal Instalasi Docker (Sebelum Setup Github)
2.9 - 2.12	Inti	Spesifikasi tugas: - Instalasi Git, - Setup repo di GitHub(private) - Deploy key - Cloning repo di VM

Note: Bagian 2.1–2.8 berikut ini merupakan langkah tambahan sebagai persiapan awal sebelum proses setup Git dilakukan.

2.1. Update dan Instalasi prerequisite

Perintah yang digunakan:

```
sudo apt update
```

```
sudo apt install -y ca-certificates curl gnupg lsb-release
```

```
git -- version
```

Langkah-langkah dan Penjelasan:

1. **sudo apt update** dipakai untuk refresh daftar paket di sistem, supaya APT tahu versi terbaru dari semua software yang tersedia.
2. **sudo apt install -y ca-certificates curl gnupg lsb-release** akan langsung menginstalasi empat paket penting yang mencakup:
 - a. **ca-certificate:** memvalidasi koneksi HTTPS dengan aman
 - b. **curl:** membuat ambil data dari internet lewat terminal
 - c. **gnupg:** handle tanda tangan digital dan enkripsi
 - d. **lsb-release:** ngambil info soal versi Ubuntu yang dipakai, berguna waktu nulis konfigurasi repo biar cocok dengan distro yang ada.

Screenshot:

```
larashtm@larasVM: ~$ sudo apt update
sudo apt install -y ca-certificates curl gnupg lsb-release
Get:1 https://download.docker.com/linux/ubuntu jammy InRelease [48.8 kB]
Hit:2 http://azure.archive.ubuntu.com/ubuntu jammy InRelease
Hit:3 http://azure.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:4 http://azure.archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:5 http://azure.archive.ubuntu.com/ubuntu jammy-security InRelease
Err:1 https://download.docker.com/linux/ubuntu jammy InRelease
  The following signatures couldn't be verified because the public key is not available: NO_PUBKEY 7EA0A9C3F273FCD8
Reading package lists... Done
W: GPG error: https://download.docker.com/linux/ubuntu jammy InRelease: The following signatures couldn't be verified because the public key is not available: NO_PUBKEY 7EA0A9C3F273FCD8
E: The repository 'https://download.docker.com/linux/ubuntu jammy InRelease' is not signed.
N: Updating from such a repository can't be done securely, and is therefore disabled by default.
N: See apt-secure(8) manpage for repository creation and user configuration details.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
lsb-release is already the newest version (11.1.0ubuntu4).
lsb-release set to manually installed.
ca-certificates is already the newest version (20240203~22.04.1).
curl is already the newest version (7.81.0-1ubuntu1.20).
gnupg is already the newest version (2.2.27-3ubuntu2.3).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
larashtm@larasVM:~$
```

2.2. Menambah GPG key Docker

Perintah yang digunakan:

```
sudo mkdir -p /etc/apt/keyrings
```

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg  
--dearmor -o /etc/apt/keyrings/docker.gpg
```

Langkah-langkah dan Penjelasan:

1. Pertama, **sudo mkdir -p /etc/apt/keyrings** dipakai untuk membuat folder khusus bernama keyrings di dalam /etc/apt. Folder ini fungsinya untuk

nyimpan file GPG key, dan opsi -p itu biar nggak error kalau foldernya sudah ada sebelumnya.

2. Lalu, `curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg` akan:
 - a. Download GPG key Docker dari internet (via **curl**)
 - b. Lalu langsung dikonversi dari format ASCII-armored ke binary pake **gpg --dearmor**
 - c. Hasilnya akan disimpan ke file **docker.gpg** di dalam folder keyrings tadi

Screenshot:

```
larashtm@larasVM:~$ sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor
-o /etc/apt/keyrings/docker.gpg
File '/etc/apt/keyrings/docker.gpg' exists. Overwrite? (y/N) y
```

2.3. Menambah repo Docker

Perintah yang digunakan:

```
echo \ "deb [arch=$(dpkg --print-architecture)
signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu \ $(lsb_release -cs) stable"
| \ sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

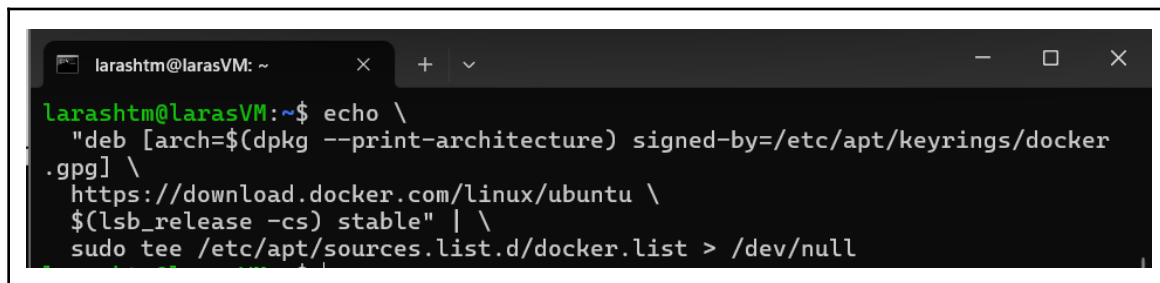
Langkah-langkah dan Penjelasan:

Perintah ini dipakai untuk menambah alamat repo resmi dari Docker ke sistem Apt agar bisa APT bisa menemukan dan menginstall paket Docker dengan benar. Lengkapnya:

Perintah **echo** ini ngebentuk satu baris konfigurasi repository yang berisi:

- Arsitektur sistem **\$(dpkg --print-architecture)**
 - Lokasi GPG key **signed-by=/etc/apt/keyrings/docker.gpg**
 - Alamat repository Docker
(**https://download.docker.com/linux/ubuntu**)
 - Dan codename dari sistem Ubuntu (**\$(lsb_release -cs)**)
1. Hasil dari ini akan dikirim ke sudo tee | \ **sudo tee**
/etc/apt/sources.list.d/docker.list > /dev/null yang akan disimpan teks tersebut ke file baru ke docker.list di dalam **etc**. File nya akan diisi dengan daftar sumber software yang bisa ditambah dengan APT.
 2. Bagian **> /dev/null** di akhir ini akan membersihkan output, agar tidak muncul di teks terminal.

Screenshot:



```
larashtm@larasVM: ~$ echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker
.gpg] \
https://download.docker.com/linux/ubuntu \
$(lsb_release -cs) stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

2.4. Update Ulang APT

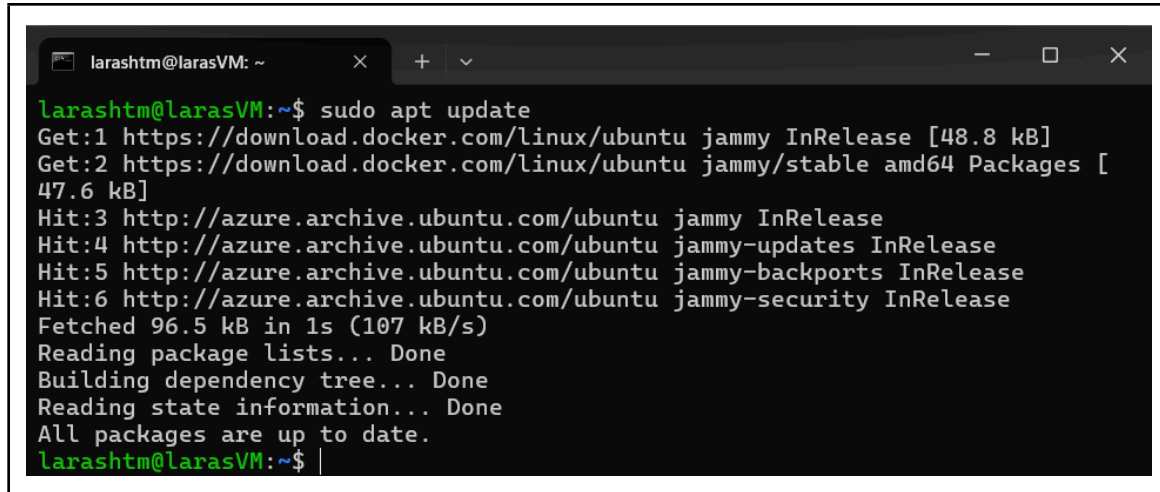
Perintah yang digunakan:

sudo apt update

Langkah-langkah dan Penjelasan:

Perintah ini dipakai untuk **refresh database APT**, jadi sistem akan mengecek semua repository yang sudah terdaftar (termasuk Docker yang barusan ditambahin) dan mengambil daftar paket terbaru dari sana. Tujuannya biar APT tahu apa saja paket yang tersedia, versi terbaru mereka, dan dari repo mana harus diambil.

Screenshot:



```
larashtm@larasVM: ~$ sudo apt update
Get:1 https://download.docker.com/linux/ubuntu jammy InRelease [48.8 kB]
Get:2 https://download.docker.com/linux/ubuntu jammy/stable amd64 Packages [47.6 kB]
Hit:3 http://azure.archive.ubuntu.com/ubuntu jammy InRelease
Hit:4 http://azure.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:5 http://azure.archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:6 http://azure.archive.ubuntu.com/ubuntu jammy-security InRelease
Fetched 96.5 kB in 1s (107 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
larashtm@larasVM:~$
```

2.5. Instalasi Docker

Perintah yang digunakan:

```
sudo apt install -y docker-ce docker-ce-cli containerd.io  
docker-buildx-plugin docker-compose-plugin  
  
docker --version
```

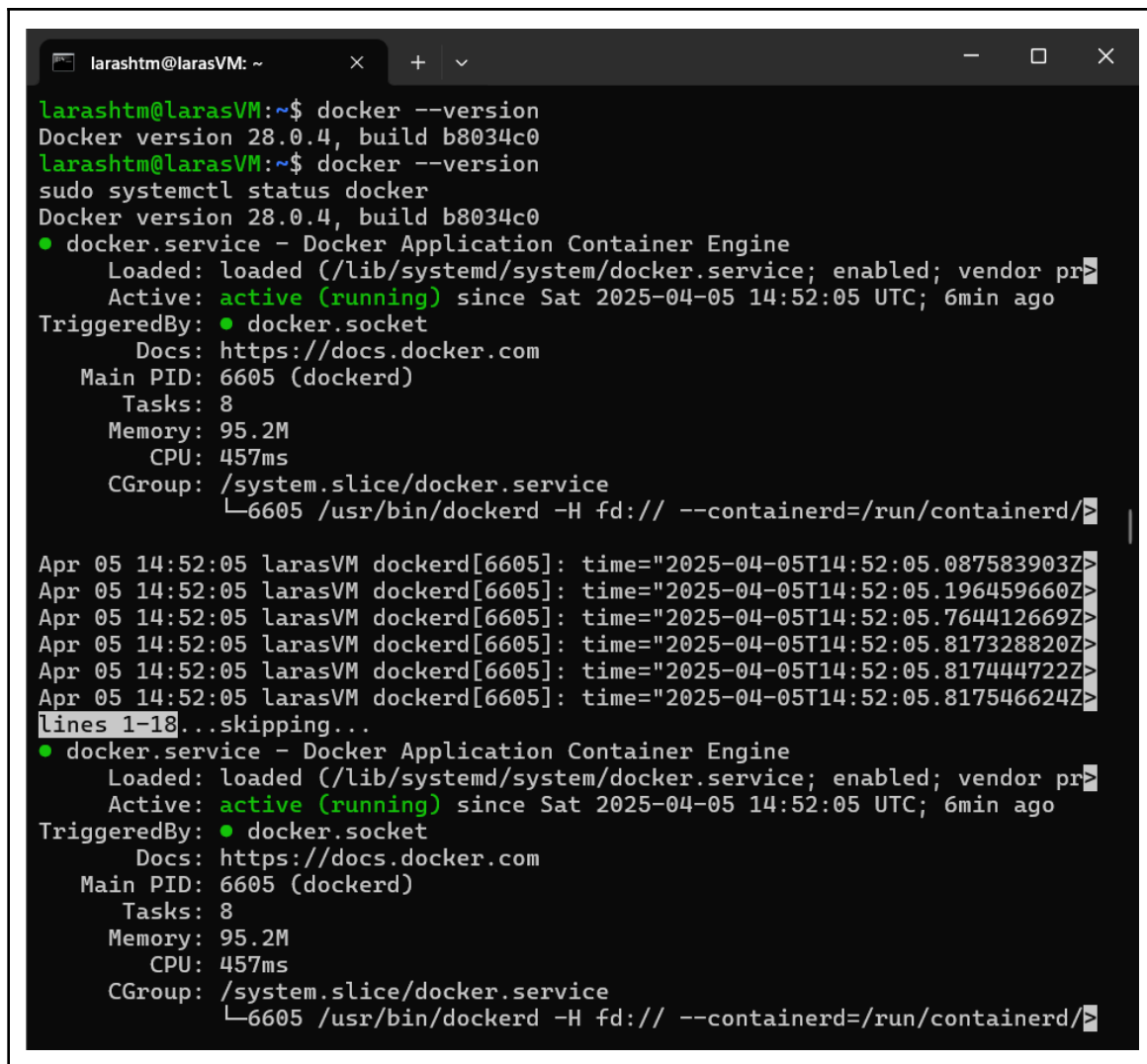
Langkah-langkah dan Penjelasan:

1. **sudo apt install -y docker-ce docker-ce-cli containerd.io**
docker-buildx-plugin docker-compose-plugin dipakai untuk menginstal Docker. Paket **docker-ce** adalah Docker Engine-nya (inti dari Docker),

docker-ce-cli adalah command-line interface-nya, containerd.io adalah runtime untuk menjalankan container, dan dua plugin lainnya (buildx dan compose) menambahkan fitur untuk build image dan mengatur multi-container project dengan docker compose.

2. **docker --version** adalah perintah yang digunakan untuk menunjukkan versi Docker yang terpasang.

Screenshot:



```
larashtn@larasVM: ~  
larashtn@larasVM:~$ docker --version  
Docker version 28.0.4, build b8034c0  
larashtn@larasVM:~$ docker --version  
sudo systemctl status docker  
Docker version 28.0.4, build b8034c0  
● docker.service - Docker Application Container Engine  
   Loaded: loaded (/lib/systemd/system/docker.service; enabled; vendor pr  
   Active: active (running) since Sat 2025-04-05 14:52:05 UTC; 6min ago  
   TriggeredBy: ● docker.socket  
     Docs: https://docs.docker.com  
    Main PID: 6605 (dockerd)  
      Tasks: 8  
     Memory: 95.2M  
        CPU: 457ms  
    CGroup: /system.slice/docker.service  
            └─6605 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/
```

Apr 05 14:52:05 larasVM dockerd[6605]: time="2025-04-05T14:52:05.087583903Z"
Apr 05 14:52:05 larasVM dockerd[6605]: time="2025-04-05T14:52:05.196459660Z"
Apr 05 14:52:05 larasVM dockerd[6605]: time="2025-04-05T14:52:05.764412669Z"
Apr 05 14:52:05 larasVM dockerd[6605]: time="2025-04-05T14:52:05.817328820Z"
Apr 05 14:52:05 larasVM dockerd[6605]: time="2025-04-05T14:52:05.817444722Z"
Apr 05 14:52:05 larasVM dockerd[6605]: time="2025-04-05T14:52:05.817546624Z"
lines 1-18...skipping...
● docker.service - Docker Application Container Engine
 Loaded: loaded (/lib/systemd/system/docker.service; enabled; vendor pr
 Active: active (running) since Sat 2025-04-05 14:52:05 UTC; 6min ago
 TriggeredBy: ● docker.socket
 Docs: https://docs.docker.com
 Main PID: 6605 (dockerd)
 Tasks: 8
 Memory: 95.2M
 CPU: 457ms
 CGroup: /system.slice/docker.service
 └─6605 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/

```
larashtm@larasVM: ~  
Apr 05 14:52:05 larasVM dockerd[6605]: time="2025-04-05T14:52:05.087583903Z">  
Apr 05 14:52:05 larasVM dockerd[6605]: time="2025-04-05T14:52:05.196459660Z">  
Apr 05 14:52:05 larasVM dockerd[6605]: time="2025-04-05T14:52:05.764412669Z">  
Apr 05 14:52:05 larasVM dockerd[6605]: time="2025-04-05T14:52:05.817328820Z">  
Apr 05 14:52:05 larasVM dockerd[6605]: time="2025-04-05T14:52:05.817444722Z">  
Apr 05 14:52:05 larasVM dockerd[6605]: time="2025-04-05T14:52:05.817546624Z">  
Apr 05 14:52:05 larasVM dockerd[6605]: time="2025-04-05T14:52:05.899941305Z">  
Apr 05 14:52:05 larasVM dockerd[6605]: time="2025-04-05T14:52:05.914486467Z">  
Apr 05 14:52:05 larasVM dockerd[6605]: time="2025-04-05T14:52:05.915321082Z">  
Apr 05 14:52:05 larasVM systemd[1]: Started Docker Application Container En>  
~  
~  
~  
~  
~  
~  
~  
lines 12-22/22 (END)
```

Catatan:

Q (quit) supaya balikin langsung ke mode terminal VM biasa yang bisa diketik lagi.

2.6. Memastikan docker berjalan

Perintah yang digunakan:

```
sudo systemctl enable docker
```

```
sudo systemctl start docker
```

```
sudo systemctl status docker
```

Langkah-langkah dan Penjelasan:

1. **sudo systemctl enable docker**

Perintah ini dipakai untuk **mengaktifkan Docker secara otomatis saat sistem dinyalakan** (booting). Jadi tiap kali me-restart laptop/komputermu, Docker bakal langsung nyala tanpa harus start manual lagi.

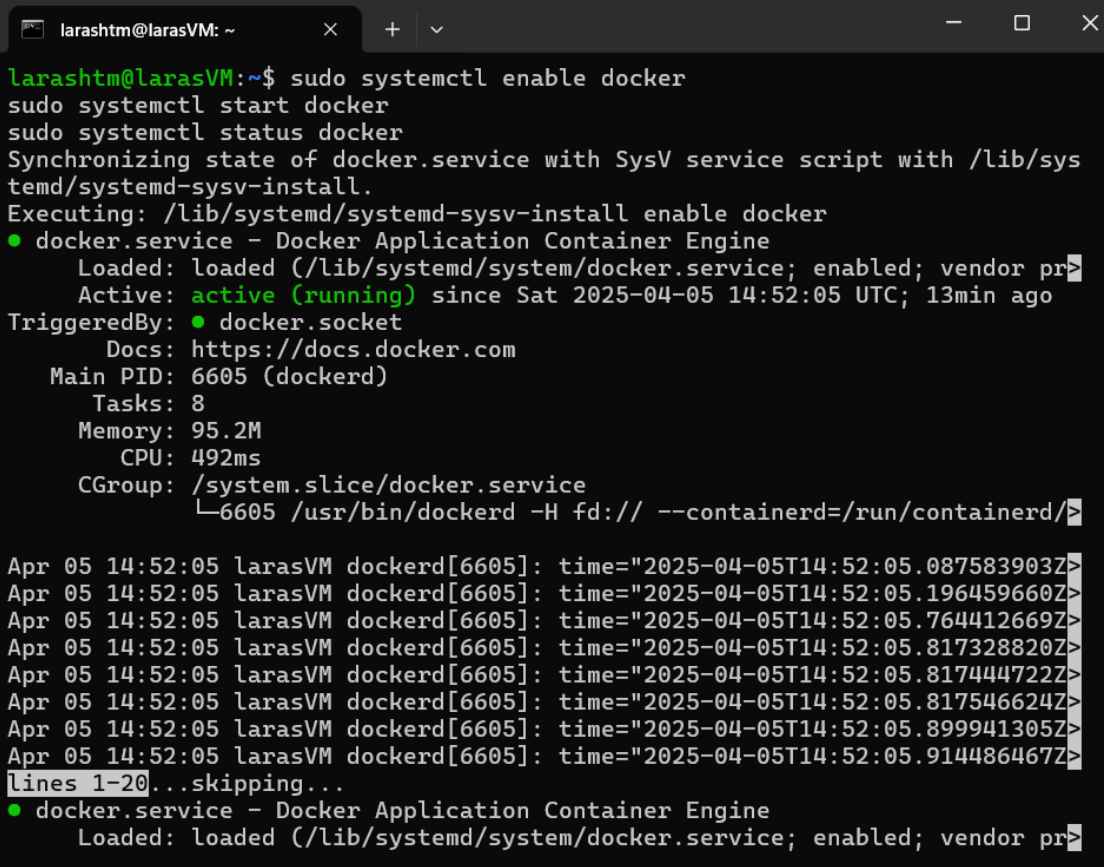
2. **sudo systemctl start docker**

Untuk **mulai menjalankan layanan Docker sekarang juga**. Tanpa perintah ini, Docker belum tentu aktif meskipun udah di-install.

3. **sudo systemctl status docker**

Ini untuk **ngecek status layanan Docker**, apakah sedang aktif (running), gagal, atau berhenti. Biasanya output-nya bakal ngasih info kalau Docker berhasil jalan, lengkap dengan PID dan status lainnya.

Screenshot:



```
larashtm@larasVM: ~  
larashtm@larasVM:~$ sudo systemctl enable docker  
sudo systemctl start docker  
sudo systemctl status docker  
Synchronizing state of docker.service with SysV service script with /lib/sys  
temd/systemd-sysv-install.  
Executing: /lib/systemd/systemd-sysv-install enable docker  
● docker.service - Docker Application Container Engine  
   Loaded: loaded (/lib/systemd/system/docker.service; enabled; vendor pr  
   Active: active (running) since Sat 2025-04-05 14:52:05 UTC; 13min ago  
TriggeredBy: ● docker.socket  
   Docs: https://docs.docker.com  
  Main PID: 6605 (dockerd)  
    Tasks: 8  
  Memory: 95.2M  
     CPU: 492ms  
   CGroup: /system.slice/docker.service  
           └─6605 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/
```

Apr 05 14:52:05 larasVM dockerd[6605]: time="2025-04-05T14:52:05.087583903Z">
Apr 05 14:52:05 larasVM dockerd[6605]: time="2025-04-05T14:52:05.196459660Z">
Apr 05 14:52:05 larasVM dockerd[6605]: time="2025-04-05T14:52:05.764412669Z">
Apr 05 14:52:05 larasVM dockerd[6605]: time="2025-04-05T14:52:05.817328820Z">
Apr 05 14:52:05 larasVM dockerd[6605]: time="2025-04-05T14:52:05.817444722Z">
Apr 05 14:52:05 larasVM dockerd[6605]: time="2025-04-05T14:52:05.817546624Z">
Apr 05 14:52:05 larasVM dockerd[6605]: time="2025-04-05T14:52:05.8999941305Z">
Apr 05 14:52:05 larasVM dockerd[6605]: time="2025-04-05T14:52:05.914486467Z">
lines 1-20...skipping...
● docker.service - Docker Application Container Engine
 Loaded: loaded (/lib/systemd/system/docker.service; enabled; vendor pr

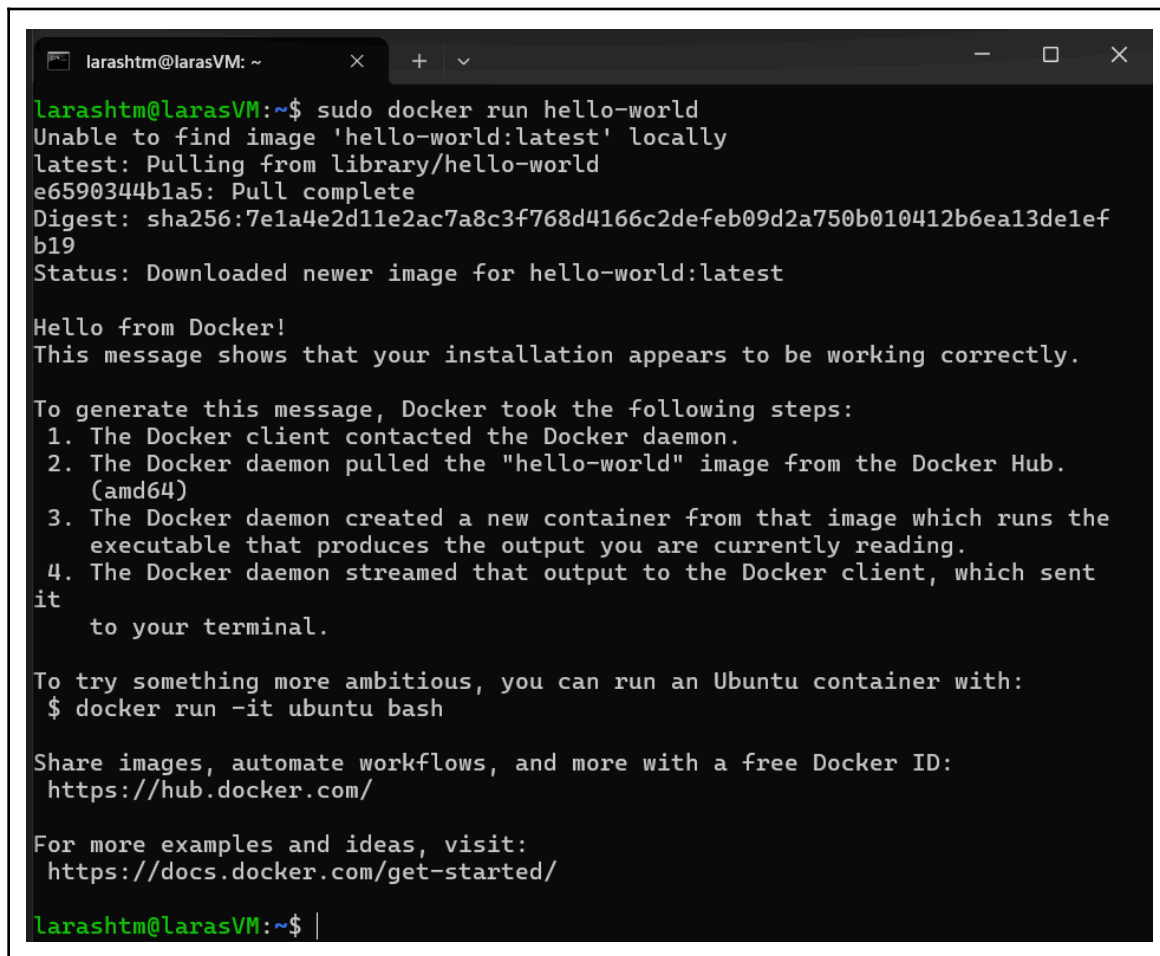
2.7. Menjalankan contoh container

sudo docker run hello-world

Langkah-langkah dan Penjelasan:

Perintah **sudo docker run hello-world** digunakan untuk mengetes apakah Docker sudah terinstal dan berjalan dengan benar. Docker akan mendownload image hello-world dari Docker Hub (jika belum ada), lalu menjalankan container sederhana yang menampilkan pesan sambutan.

Screenshot:

A screenshot of a terminal window titled 'larashtm@larasVM: ~'. The terminal shows the command 'sudo docker run hello-world' being executed. The output indicates that the 'hello-world:latest' image was not found locally and was pulled from the Docker Hub. The terminal then displays a 'Hello from Docker!' message and a detailed list of steps Docker took to generate the message. It also provides instructions on how to run an Ubuntu container and links to Docker's website for more information. The prompt 'larashtm@larasVM:~\$' is visible at the bottom of the terminal.

```
larashtm@larasVM:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
e6590344b1a5: Pull complete
Digest: sha256:7e1a4e2d11e2ac7a8c3f768d4166c2defeb09d2a750b010412b6ea13de1efb19
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent
    it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

larashtm@larasVM:~$ |
```

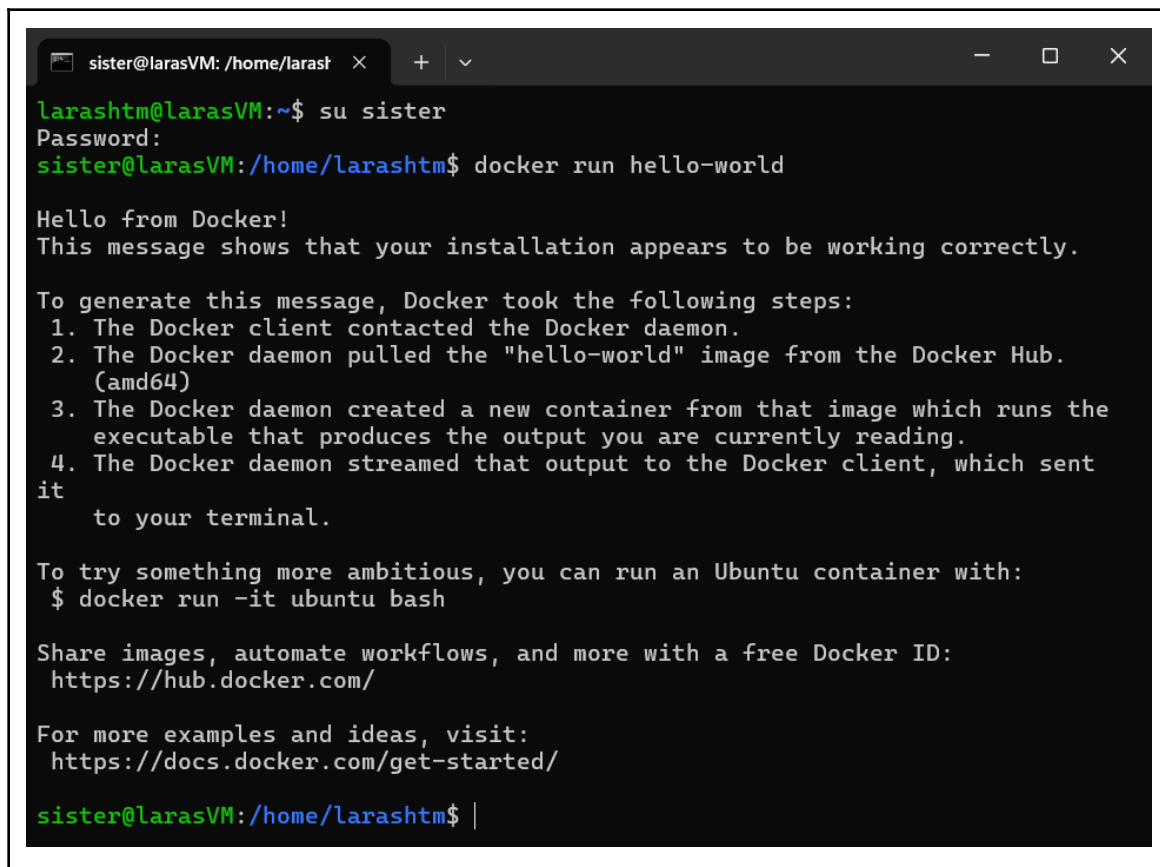
2.8. Me-run “Hello World”

docker run hello-world

Langkah-langkah dan Penjelasan:

Perintah docker run **hello-world** dipakai untuk ngetes apakah user sudah bisa jalanin **Docker** tanpa sudo. Kalau perintah ini berhasil dan muncul pesan “**Hello from Docker!**”, berarti user sudah masuk ke grup docker dan punya izin penuh untuk menjalankan container. Tapi kalau masih muncul error “permission denied”, logout dari sesi sekarang, terus login ulang biar perubahan grupnya aktif.

Screenshot:



```
sister@larasVM: /home/larasht  
larasht@larasVM:~$ su sister  
Password:  
sister@larasVM:/home/larasht$ docker run hello-world  
  
Hello from Docker!  
This message shows that your installation appears to be working correctly.  
  
To generate this message, Docker took the following steps:  
1. The Docker client contacted the Docker daemon.  
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.  
   (amd64)  
3. The Docker daemon created a new container from that image which runs the  
   executable that produces the output you are currently reading.  
4. The Docker daemon streamed that output to the Docker client, which sent  
   it  
   to your terminal.  
  
To try something more ambitious, you can run an Ubuntu container with:  
$ docker run -it ubuntu bash  
  
Share images, automate workflows, and more with a free Docker ID:  
https://hub.docker.com/  
  
For more examples and ideas, visit:  
https://docs.docker.com/get-started/  
  
sister@larasVM:/home/larasht$ |
```


2.9. Instalasi GIT di VM

Perintah yang digunakan:

```
sudo apt update
```

```
sudo apt install git
```

```
git --version
```

Langkah-langkah dan Penjelasan:

1. Dua perintah teratas digunakan untuk **menyiapkan dan menginstal Git di sistem**. Pertama, **sudo apt update** digunakan untuk menyegarkan daftar paket agar APT tahu versi terbaru dari semua software, termasuk Git. Setelah itu, **sudo apt install git** akan menginstal Git, yaitu alat penting untuk mengelola versi kode dan nge-clone repositori dari GitHub atau platform sejenis
2. **git --version** untuk mengecek versi git

Screenshot:

```
larashtm@larasVM: ~$ sudo apt update
sudo apt install -y ca-certificates curl gnupg lsb-release
Get:1 https://download.docker.com/linux/ubuntu jammy InRelease [48.8 kB]
Hit:2 http://azure.archive.ubuntu.com/ubuntu jammy InRelease
Hit:3 http://azure.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:4 http://azure.archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:5 http://azure.archive.ubuntu.com/ubuntu jammy-security InRelease
Err:1 https://download.docker.com/linux/ubuntu jammy InRelease
  The following signatures couldn't be verified because the public key is not available: NO_PUBKEY 7EA0A9C3F273FCD8
Reading package lists... Done
W: GPG error: https://download.docker.com/linux/ubuntu jammy InRelease: The following signatures couldn't be verified because the public key is not available: NO_PUBKEY 7EA0A9C3F273FCD8
E: The repository 'https://download.docker.com/linux/ubuntu jammy InRelease' is not signed.
N: Updating from such a repository can't be done securely, and is therefore disabled by default.
N: See apt-secure(8) manpage for repository creation and user configuration details.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
lsb-release is already the newest version (11.1.0ubuntu4).
lsb-release set to manually installed.
ca-certificates is already the newest version (20240203~22.04.1).
curl is already the newest version (7.81.0-1ubuntu1.20).
gnupg is already the newest version (2.2.27-3ubuntu2.3).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
larashtm@larasVM:~$ |

laras17@larasVM:/home/larashtm$ git --version
git version 2.34.1
laras17@larasVM:/home/larashtm$ |
```

2.10. Membuat Repository Baru di GitHub

Langkah-langkah dan Penjelasan:

1. **Buka template yang telah disediakan** oleh admin pada file Tugas Kecil 2 di Google Docs.
2. Dari file tersebut, **klik tautan menuju template** repository yang sudah disiapkan oleh asisten.
3. Setelah halaman template terbuka, klik tombol **“Use this template”**.
4. Selanjutnya, akan muncul tampilan **Create a new repository**.
5. Beri nama repository sesuai ketentuan, misalnya: II2210__Tucil 2.
6. Pastikan **repository bersifat private**, sesuai dengan spesifikasi yang telah ditentukan oleh asisten.
7. Klik **Create repository** untuk menyelesaikan proses pembuatan.

Screenshot:

CodeIssuesPull requestsActionsProjectsSecurityInsights

SMS 2FA method is prone to fraud and may be unreliable as it depends on delivery success rates in your region. To ensure consistent access to your account, please configure an additional 2FA method.

View 2FA settings

ii2210-2025-template-tucilPublic template

WatchForkStarUse this templateCreate a new repositoryOpen in a codespace

main1 Branch0 Tags

Go to file

Add fileCode

About

plumtrainerfrost

fic_model.py

Initial commit

2 weeks ago

Dockerfile

feat: tester.py

2 weeks ago

main.py

fic_model.py

last week

model.py

feat: tester.py

2 weeks ago

tester.py

fic_model.py

2 weeks ago

Template untuk Tugas Kecil 2 II2210.

Activity

Custom properties

0 stars

0 watching

0 forks

Report repository

Releases

No releases published

Packages

No packages published

Languages

Python 100.0%

© 2025 GitHub, Inc.

TermsPrivacySecurityStatusDocsContactManage cookiesDo not share my personal information

https://github.com/new?template_name=ii2210-2025-template-tucil&template_owner=Lab-Sister

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere?
[Import a repository.](#)

Required fields are marked with an asterisk (*).

Repository template

Lab-Sister/ii2210-2025-template-tucil

Start your repository with a template repository's contents.

☐ Include all branches

Copy all branches from Lab-Sister/ii2210-2025-template-tucil and not just the default branch.

Owner *Repository name *

larashtm / II2210_Tucil 2

Your new repository will be created as II2210_Tucil-2.

The repository name can only contain ASCII letters, digits, and the characters -, ., and _.

Great repository names are short and memorable. Need inspiration? How about curly-dollop ?

Description (optional)

Public

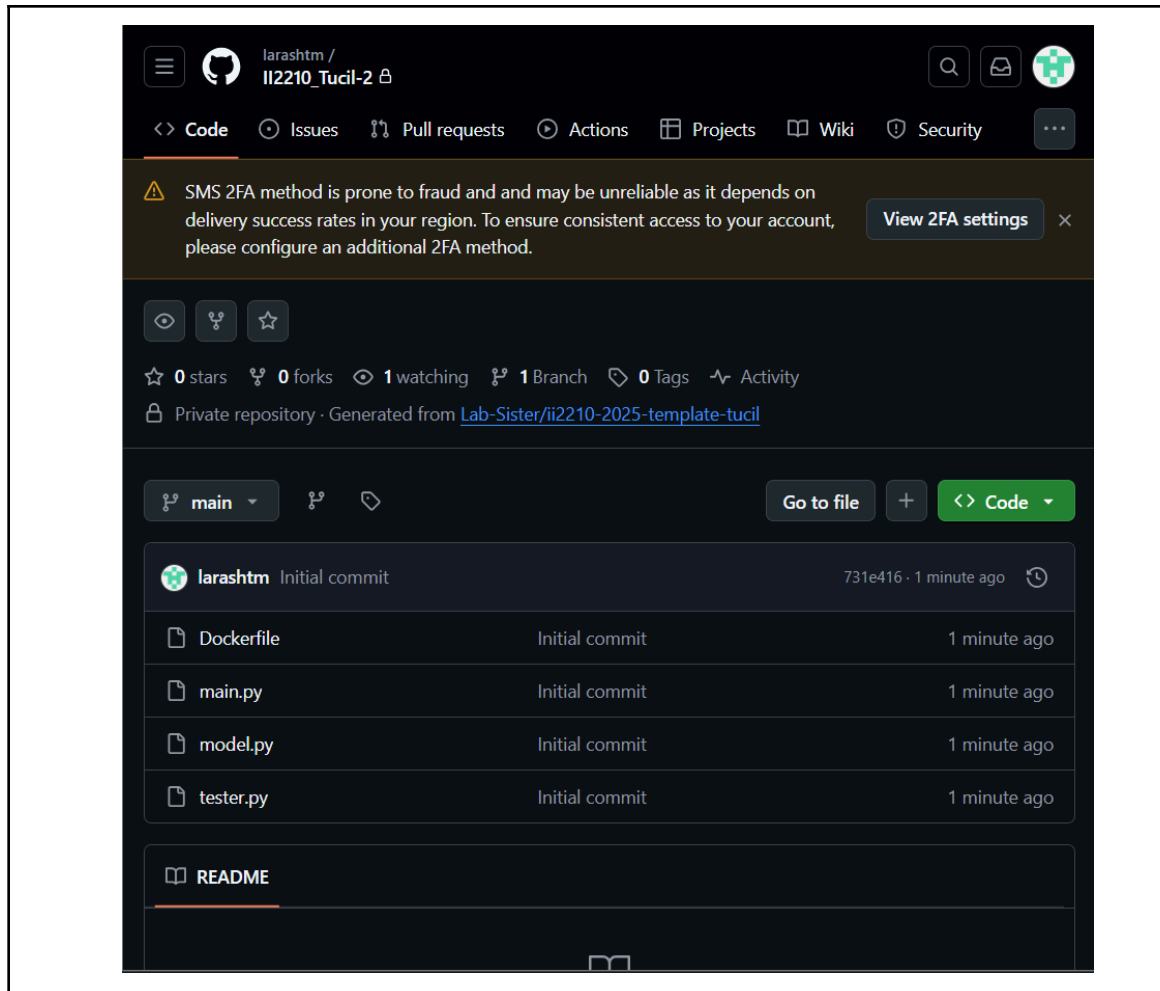
Anyone on the internet can see this repository. You choose who can commit.

Private

You choose who can see and commit to this repository.

You are creating a private repository in your personal account.

Create repository



2.11. Menambahkan Deploy Key ke Repository

Perintah yang digunakan:

```
C:\Users\laras\.ssh>type %USERPROFILE%\.ssh\id_rsa.pub  
cat ~/.ssh/id_rsa.pub
```

Langkah-langkah dan Penjelasan:

Perintah yang digunakan untuk memunculkan rsa yang nantinya akan diperlukan untuk menambahkan ke GitHub. Kurang lebih tampilannya sebagai berikut:

ssh-rsa

AAAAB3NzaC1yc2EAAAADAQABAAQCAQDFQpnosS5lvqIkFXFOCwVHT
A700anIihC9+P9dCO3wKCnllJ4XLMtDfA1Fsxkq70yuOaIyx2t/q/eiTV5H
ElHaF25edTvcG20+iX7Oedg8Wo/pbNrdTLmxttf6F6GmjwMpHKiebkqJg7
NC9nft+PXktFkU8cTeDaz9X/0mJr5LFDJ0qhtSKBJKq1MsBOPGZLIWMiO
6NufgC4CoNg8Rd3DKhYdPae6Q4GSF1Kv0ZjF5NA6KuPH0Ny001wFc9J2
VTUyNuhvpc08duJETpa50bJ+/JtOL3jsPSY8fXUUsb33r08fBO4wktCSJB
mgU1mZ43+cqZ3/H09dXzghiFh2nb8T6nJLb2RCXw4F/fZTjavwT42utddo
TpDx09cnbfCZFIJ2elOU1caty1uk8y+fvf8qMOtVWFR5HVoV2oaS3EAwb6
rOC8otYUL6CVcG1otUwsK6RQqUegOJtQZhdfYiN+1NhvSc4hi3D7GMAVU
Egag9n0+NCzYBYKCtvahz7tRb6BdnExeq4NMXQnz07DPdys8d3tCVIHN
3W2B60yx6yCGx7wHMhwyxUfspg/xzjtnleJ6ASSVeawBYAlhxcNhaeMw
MDDRouqwr/gtxXakq3vTiykUE5aVAMiLSXmk8PtyMv9tDvowFUmQ0hl4p
ahHaqrMDPgYOOALfOYRxNo/4Ca0HdNw==

larashtm@users.noreply.github.com

1. Setelah berhasil, masuk ke GitHub dan tambahkan deploy key agar responya bisa menambahkan deploy key supaya dapat diakses dari virtual machine.
2. Klik “Add deploy key”
3. Pada kolom Title dapat diisi sesuai kebutuhan tugas. Misal:”Vm Azure”.
4. Pada kolom Key, masukkan public key yang dihasilkan sebelumnya di terminal dimulai dengan **ssh-rsa**.
5. Pastikan opsi “Allow write access” tidak tercentang lalu klik “Add Key”

Screenshot:

```

-rw----- 1 larashtm larashtm 3401 Apr  6 10:20 id_rsa
-rw-r--r-- 1 larashtm larashtm 759 Apr  6 10:20 id_rsa.pub
-rw----- 1 larashtm larashtm 1120 Apr  5 23:49 known_hosts
-rw-r--r-- 1 larashtm larashtm 142 Mar 26 06:35 known_hosts.old
larashtm@larasVM:~/II2210_Tucil-2/II2210_Tucil-2$ cat ~/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQDAQDFQpnosS5lvqIkFXFOCwVHTA700anIihC9+P9d
CO3wKcNllJ4XLMtDfA1Fsxkq70yu0aIyx2t/q/eiTV5HELHaF25edTvcG20+iX70edg8Wo/pbNrd
TLmxttf6F6GmjwMphKiebkqJg7NC9nft+PXktFkU8cTeDaz9X/0mJr5LFDJ0qhtSKBJKq1MsBOPG
ZLLWmi06NufgC4CoNg8Rd3DKhYdPae6Q4GSF1Kv0ZjF5NA6KuPH0Ny001wFc9J2VTUyNuhvpc08d
uJETpa50bJ+/JtOL3jsPSY8fXUUsb33r08fBO4wktCSJBmgU1mZ43+cqZ3/H09dXzghiFh2nb8T6
nJLb2RCXw4F/fZTjavwT42utddoTpDx09cnbfCZFIJ2eLOU1caty1uk8y+fVf8qM0tVWFR5HVoV2
oaS3EAwb6rOC8otYUL6CVcG1otUwsK6RQqUeg0JtQZhdFYiN+1NhvSc4hi3D7GMaVUEgag9n0+NC
zYBYKctvahz7tRb6BdnExeq4NMxQnz07DPdys8d3tCVIHN3W2B60yx6yCGx7wHMHwyUfsgp/xzj
tnleJ6ASSVeawBYAlhxcNhaeMwMDDRouqwr/gtxXakq3vTiykUE5aVAMiLSXmk8PtyMv9tDvowFU
mq0hl4pahHaqrMDPgY00ALfOYRxNo/4Ca0HdNw== larashtm@users.noreply.github.com
larashtm@larasVM:~/II2210_Tucil-2/II2210_Tucil-2$ git clone git@github.com:

```

General

Access

Collaborators

Code and automation

Branches

Tags

Rules

Actions

Webhooks

Environments

Codespaces

Pages

Security

Advanced Security

Deploy keys

Secrets and variables

Deploy keys / Add new

Title

vm azure

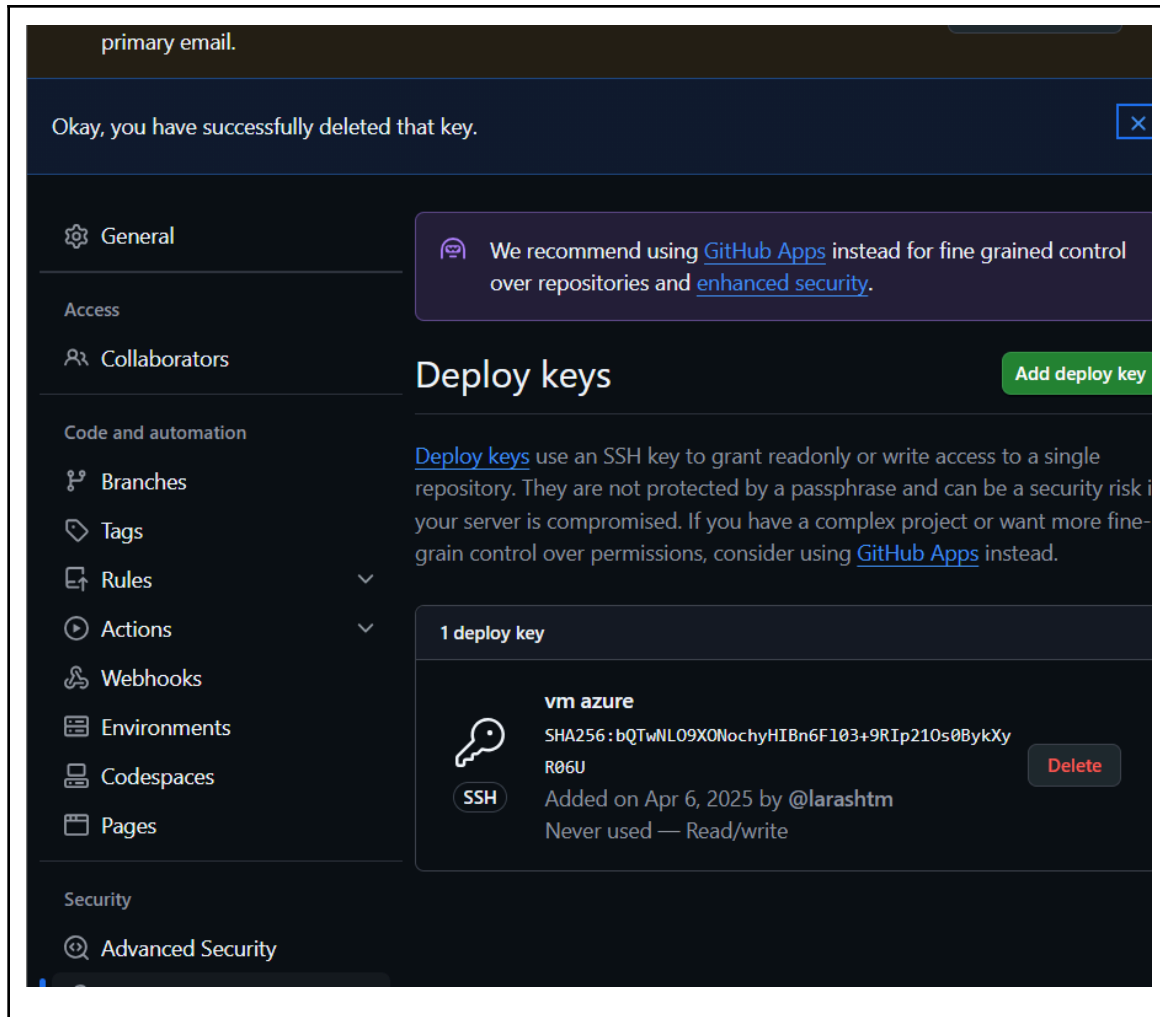
Key

ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQDAQDFQpnosS5lvqIkFXFOCwVHTA7
00anIihC9+P9dCO3wKcNllJ4XLMtDfA1Fsxkq70yu0aIyx2t/q/eiTV5HELHaF
25edTvcG20+iX7Oedg8Wo/pbNrdTLmxttf6F6GmjwMphKiebkqJg7NC9n
ft+PXktFkU8cTeDaz9X/0mJr5LFDJ0qhtSKBJKq1MsBOPGZLLWmi06NufgC4
CoNg8Rd3DKhYdPae6Q4GSF1Kv0ZjF5NA6KuPH0Ny001wFc9J2VTUyNuh
vpc08duJETpa50bJ+/JtOL3jsPSY8fXUUsb33r08fBO4wktCSJBmgU1mZ43
+cqZ3/H09dXzghiFh2nb8T6nJLb2RCXw4F/fZTjavwT42utddoTpDx09cnbf
CZFIJ2eLOU1caty1uk8y+fVf8qM0tVWFR5HVoV2oaS3EAwb6rOC8otYUL6
CVcG1otUwsK6RQqUeg0JtQZhdFYiN+1NhvSc4hi3D7GMaVUEgag9n0+NC
zYBYKctvahz7tRb6BdnExeq4NMxQnz07DPdys8d3tCVIHN3W2B60yx6yCGx7wHMHwyUfsgp/xzj
tnleJ6ASSVeawBYAlhxcNhaeMwMDDRouqwr/gtxXakq3vTiykUE5aVAMiLSXmk8PtyMv9tDvowFU
mq0hl4pahHaqrMDPgY00ALfOYRxNo/4Ca0HdNw== larashtm@users.noreply.github.com

Begins with 'ssh-rsa', 'ecdsa-sha2-nistp256', 'ecdsa-sha2-nistp384', 'ecdsa-sha2-nistp521',
'ssh-ed25519', 'sk-ecdsa-sha2-nistp256@openssh.com', or 'sk-ssh-ed25519@openssh.com'

☒ Allow write access
Can this key be used to push to this repository? Deploy keys always have pull access.

Add key



2.12. Menambahkan Deploy Key ke Repository

Perintah yang digunakan:

```
git clone git@github.com:larashtm/II2210_Tucil-2.git (pakai SSH)
```

Langkah-langkah dan Penjelasan:

Perintah ini dipakai untuk meng-clone repo GitHub via SSH bukan HTTPS dan mengambil salinan penuh dari repo yang tadi sudah dibuat dan menyimpan di folder lokal repo. Supaya perintah ini jalan maka harus mempunyai SSH key dan Public key.

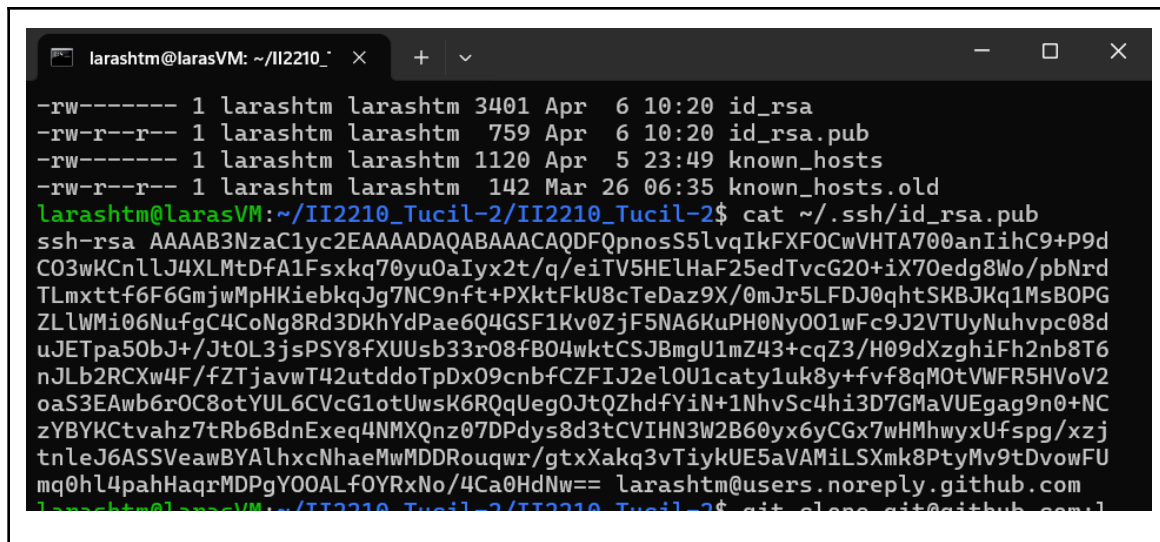
Cara membuat PAT:

- Login ke GitHub dan pilih settings
- Klik Developer settings dan pilih Personal access tokens -> Tokens (classic)
- Generate new token dan pilih classic
- Kasih nama tokennya dan pilih expiration (boleh disesuaikan)
- Scope, boleh dicentang sesuai dengan kebutuhan
- Klik Generate token

=> Dengan menggunakan PAT, maka bisa menggunakan repo tanpa masalah autentikasi. More info tentang PAT di git saya:

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

Screenshot:



```
larashtm@larasVM: ~/II2210_ x + v
-rw----- 1 larashtm larashtm 3401 Apr  6 10:20 id_rsa
-rw-r--r-- 1 larashtm larashtm  759 Apr  6 10:20 id_rsa.pub
-rw----- 1 larashtm larashtm 1120 Apr  5 23:49 known_hosts
-rw-r--r-- 1 larashtm larashtm  142 Mar 26 06:35 known_hosts.old
larashtm@larasVM:~/II2210_Tucil-2/II2210_Tucil-2$ cat ~/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQDAQDFQpnosS5lvqIkFXFOCwVHTA700anIihC9+P9d
C03wKcNllJ4XLMtDfA1Fsxkq70yu0aIyx2t/q/eiT5HELHaF25edTvcG20+iX70edg8Wo/pbNrd
TLmxttf6F6GmjwMphKiebkqJg7NC9nft+PXktFkU8cTeDaz9X/0mJr5LFDJ0qhtSKBJKq1MsB0PG
ZLLWmi06NufgC4CoNg8Rd3DKhYdPae6Q4GSF1Kv0ZjF5NA6KuPH0Ny001wFc9J2VTUyNuhvpc08d
uJETpa50bJ+/Jt0L3jsPSY8fXUUsb33r08fB04wktCSJBmgU1mZ43+cqZ3/H09dXzghiFh2nb8T6
nJLb2RCXw4F/fZTjavwT42utddoTpDx09cnbfCZFIJ2e10U1caty1uk8y+fvf8qM0tVWFR5HVoV2
oaS3EAwb6r0C8otYUL6CVcG1otUwsK6RQqUeg0JtQZhdFYiN+1NhvSc4hi3D7GmaVUEgag9n0+NC
zYBYKctvahz7tRb6BdnExeq4NMXQnz07DPdys8d3tCVIHN3W2B60yx6yCGx7wHMhwyxUfspg/xzj
tnleJ6ASSVeawBYAlhxcNhaeMwMDDRouqwr/gtxXakq3vTiykUE5aVAMiLSXmk8PtyMv9tDvowFU
mq0hl4pahHaqrMDPgY00ALfOYRxNo/4Ca0HdNw== larashtm@users.noreply.github.com
larashtm@larasVM:~/II2210_Tucil-2/II2210_Tucil-2$ git clone git@github.com:1
```


⚙️ General

Access

👤 Collaborators

Code and automation

🔑 Branches

🏷️ Tags

📄 Rules

🕒 Actions

🔗 Webhooks

📁 Environments

📄 Codespaces

📄 Pages

Security

🔍 Advanced Security

🔑 Deploy keys

🔒 Secrets and variables

Deploy keys / Add new

Title

vm azure

Key

ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQCAQDFQpnosS5lvqIkFXFOCwVHTA7
00anlihC9+P9dCO3wKcNlU4XLMtDfA1Fsxkq70yuOalyx2t/q/eiTV5HElHaF
25edTvcG2O+iX7Oedg8Wo/pbNrdTLmxttf6F6GmjwMpHKiebkqJg7NC9n
ft+PXktFkU8cTeDaz9X/0mJr5LFDJ0qhtSKBJKq1MsBOPGZLIWMI06NufgC4
CoNg8Rd3DKhYdPae6Q4GSF1Kv0ZjF5NA6KuPH0NyOO1wFc9J2VTUyNuh
vpc08dulETpa5ObJ+/JtOL3jsPSY8fXUUsb33rO8fBO4wktCSJBmgU1mZ43
+cqZ3/H09dXzghiFh2nb8T6nJLb2RCXw4F/fZTjavwT42utddoTpDxO9cnbf
CZFII2eIOU1caty1uk8y+fvf8qMOtVWFR5HVoV2oaS3EAwb6rOC8otYUL6
Q1S4M1K2PQ1H1K3ZL18M14M1S1K3P3C14M1F1S1M14

Begin with 'ssh-rsa', 'ecdsa-sha2-nistp256', 'ecdsa-sha2-nistp384', 'ecdsa-sha2-nistp521',
'ssh-ed25519', 'sk-ecdsa-sha2-nistp256@openssh.com', or 'sk-ssh-ed25519@openssh.com'

☒ Allow write access

Can this key be used to push to this repository? Deploy keys always have pull access.

Add key

primary email.

Okay, you have successfully deleted that key.

General

Access

Collaborators

Code and automation

Branches

Tags

Rules

Actions

Webhooks

Environments

Codespaces

Pages

Security

Advanced Security

We recommend using [GitHub Apps](#) instead for fine grained control over repositories and [enhanced security](#).

Deploy keys

Add deploy key

[Deploy keys](#) use an SSH key to grant readonly or write access to a single repository. They are not protected by a passphrase and can be a security risk if your server is compromised. If you have a complex project or want more fine-grained control over permissions, consider using [GitHub Apps](#) instead.

1 deploy key



vm azure

SHA256:bQTwnL09X0NochyHIBn6F103+9RIp210s0BykXy

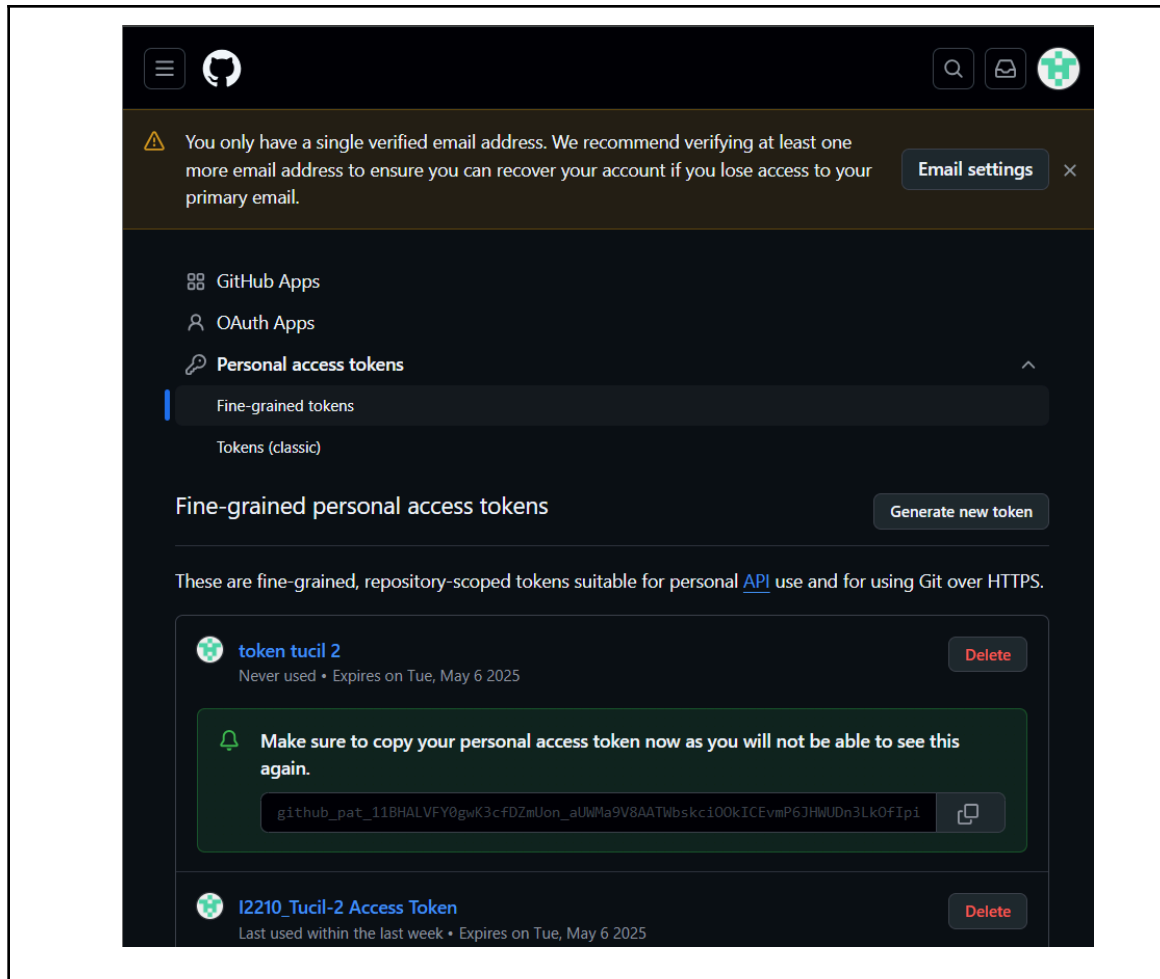
R06U

SSH

Added on Apr 6, 2025 by @larashtm

Never used — Read/write

Delete



Catatan:

Awalnya sempat mencoba metode kloning repo dengan HTTPS PAT autentikasi (Personal Access Token) mengingat GitHub tidak mendukung login pakai password. Namun pada akhirnya, proses kloning berhasil dilakukan menggunakan metode SSH, sehingga PAT tidak lagi digunakan.

3. Pembuatan Web Server Sederhana (FastAPI)

Pada bagian ini, dijelaskan secara menyeluruh proses instalasi dan pembuatan web server sederhana menggunakan framework **FastAPI**. Web server yang dibangun memiliki beberapa endpoint utama, yaitu `/` (GET) yang berfungsi untuk menampilkan file **index.html**, `/motd` (GET) yang akan menampilkan *Message of the Day* (MotD) secara acak yang disimpan dalam database SQLite, serta `/motd` (POST) yang digunakan untuk menambahkan pesan baru ke dalam database dengan proteksi autentikasi.

Langkah-langkah yang dilakukan meliputi instalasi dependensi penting menggunakan **pip**, seperti **fastapi**, **uvicorn**, dan library pendukung lainnya. Selanjutnya dilakukan pembuatan struktur file dan penulisan endpoint sesuai dengan fungsinya, dilengkapi dengan pembuatan file **Dockerfile**, **docker-compose.yml**, serta **tester.py** untuk keperluan pengujian. Seluruh tahapan didokumentasikan dengan jelas melalui tangkapan layar dan penjelasan langkah demi langkah untuk memastikan proses dapat direplikasi dan dipahami dengan baik.

3.1. Menginstalasi pip

Perintah yang digunakan:

```
sudo apt update
sudo apt install python3-pip
pip3 --version
pip install fastapi uvicorn

pip show fastapi

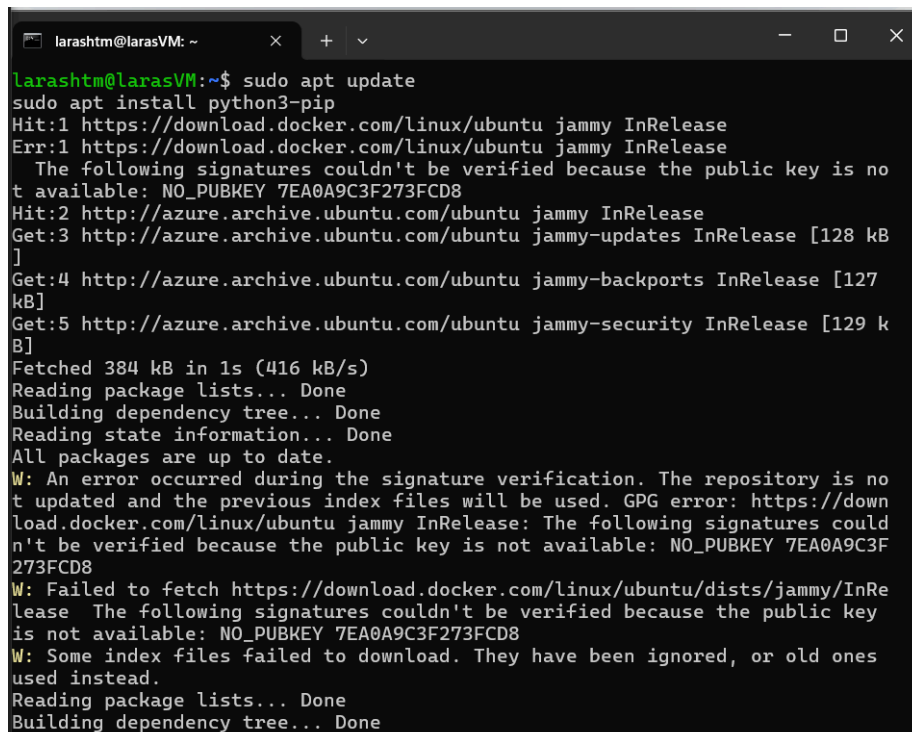
pip show uvicorn
```

Langkah-langkah dan Penjelasan:

1. Dua perintah ini dipakai untuk instalasi **pip** untuk Python 3, supaya bisa install berbagai library Python dengan mudah.

- **sudo apt update** nyegerin daftar paket supaya sistem tahu versi terbaru,
 - lalu **sudo apt install python3-pip** akan menginstal pip, package manager-nya Python 3.
2. **pip3 --version** dipakai untuk mengecek apakah sudah berhasil di install atau belum
 3. Perintah **pip install fastapi uvicorn** dipakai untuk **install FastAPI dan Uvicorn**, dua tools utama untuk bikin dan jalanin web API pake Python.
 - **fastapi** adalah framework modern untuk bikin REST API dengan Python, cepat dan mudah.
 - **uvicorn** adalah ASGI server ringan untuk menjalankan aplikasi FastAPI.
 4. Perintah **pip show fastapi** dan **pip show uvicorn** digunakan untuk menampilkan detail info dari package yang sudah di install.

Screenshot:



```
larashtm@larasVM: ~  
larashtm@larasVM:~$ sudo apt update  
sudo apt install python3-pip  
Hit:1 https://download.docker.com/linux/ubuntu jammy InRelease  
Err:1 https://download.docker.com/linux/ubuntu jammy InRelease  
  The following signatures couldn't be verified because the public key is no  
t available: NO_PUBKEY 7EA0A9C3F273FCD8  
Hit:2 http://azure.archive.ubuntu.com/ubuntu jammy InRelease  
Get:3 http://azure.archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]  
Get:4 http://azure.archive.ubuntu.com/ubuntu jammy-backports InRelease [127  
kB]  
Get:5 http://azure.archive.ubuntu.com/ubuntu jammy-security InRelease [129 k  
B]  
Fetched 384 kB in 1s (416 kB/s)  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
All packages are up to date.  
W: An error occurred during the signature verification. The repository is no  
t updated and the previous index files will be used. GPG error: https://down  
load.docker.com/linux/ubuntu jammy InRelease: The following signatures could  
n't be verified because the public key is not available: NO_PUBKEY 7EA0A9C3F  
273FCD8  
W: Failed to fetch https://download.docker.com/linux/ubuntu/dists/jammy/InRe  
lease The following signatures couldn't be verified because the public key  
is not available: NO_PUBKEY 7EA0A9C3F273FCD8  
W: Some index files failed to download. They have been ignored, or old ones  
used instead.  
Reading package lists... Done  
Building dependency tree... Done
```

```
larashtm@larasVM: ~  
Scanning linux images...  
Running kernel seems to be up-to-date.  
No services need to be restarted.  
No containers need to be restarted.  
No user sessions are running outdated binaries.  
No VM guests are running outdated hypervisor (qemu) binaries on this host.  
larashtm@larasVM:~$
```

```
larashtm@larasVM:~$ pip3 --version  
pip 22.0.2 from /usr/lib/python3/dist-packages/pip (python 3.10)
```

```
larashtm@larasVM: ~  
larashtm@larasVM:~$ pip install fastapi uvicorn  
Defaulting to user installation because normal site-packages is not writeable  
Collecting fastapi  
  Downloading fastapi-0.115.12-py3-none-any.whl (95 kB)  
    95.2/95.2 KB 1.5 MB/s eta 0:00:00  
Collecting uvicorn  
  Downloading uvicorn-0.34.0-py3-none-any.whl (62 kB)  
    62.3/62.3 KB 6.0 MB/s eta 0:00:00  
Collecting typing-extensions>=4.8.0  
  Downloading typing_extensions-4.13.1-py3-none-any.whl (45 kB)  
    45.7/45.7 KB 4.7 MB/s eta 0:00:00
```

```
larashtm@larasVM: ~  
which is not on PATH.  
  Consider adding this directory to PATH or, if you prefer to suppress this  
  warning, use --no-warn-script-location.  
  WARNING: The script fastapi is installed in '/home/larashtm/.local/bin' wh  
  ich is not on PATH.  
  Consider adding this directory to PATH or, if you prefer to suppress this  
  warning, use --no-warn-script-location.  
Successfully installed annotated-types-0.7.0 anyio-4.9.0 exceptiongroup-1.2.  
2 fastapi-0.115.12 h11-0.14.0 pydantic-2.11.2 pydantic-core-2.33.1 sniffio-1  
.3.1 starlette-0.46.1 typing-extensions-4.13.1 typing-inspection-0.4.0 uvico  
rn-0.34.0  
larashtm@larasVM:~$
```

```
larashtm@larasVM: ~$ pip show fastapi
pip show uvicorn
Name: fastapi
Version: 0.115.12
Summary: FastAPI framework, high performance, easy to learn, fast to code, ready for production
Home-page:
Author:
Author-email: =?utf-8?q?Sebasti=C3=A1n_Ram=C3=ADrez?= <tiangolo@gmail.com>
License:
Location: /home/larashtm/.local/lib/python3.10/site-packages
Requires: pydantic, starlette, typing-extensions
Required-by:
Name: uvicorn
Version: 0.34.0
Summary: The lightning-fast ASGI server.
Home-page:
Author:
Author-email: Tom Christie <tom@tomchristie.com>, Marcelo Trylesinski <marcelotryle@gmail.com>
License: BSD-3-Clause
Location: /home/larashtm/.local/lib/python3.10/site-packages
Requires: click, h11, typing-extensions
Required-by:
```

3.2. Melakukan Cloning Repo

Perintah yang digunakan:

```
git clone https://github.com/larashtm/II2210_Tucil-2.git TucilBackup
```

Langkah-langkah dan Penjelasan:

Perintah ini dipakai untuk meng-clone dari github ke folder khusus.

Screenshot:

```
larashtm@larasVM: ~/II2210_Tucil-2$ git clone https://github.com/larashtm/II2210_Tucil-2.git TucilBackup
Cloning into 'TucilBackup'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 6 (delta 0), reused 2 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (6/6), done.
larashtm@larasVM: ~/II2210_Tucil-2$
```

3.3. Melengkapi struktur file

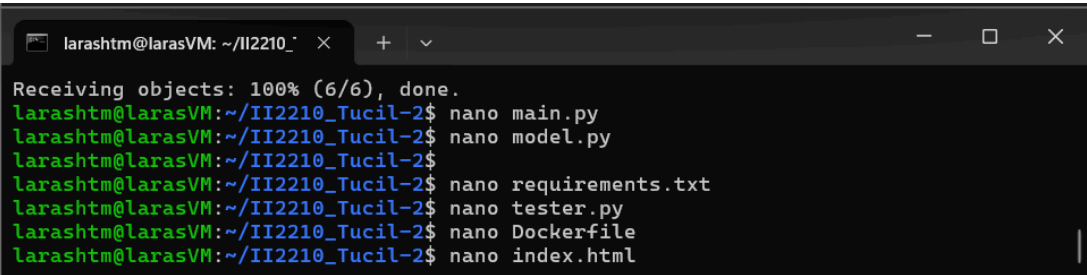
Perintah yang digunakan:

```
nano main.py
nano model.py
nano requirements.txt
nano tester.py
nano Dockerfile
nano index.html
```

Langkah-langkah dan Penjelasan:

Perintah ini digunakan untuk membuka di nano terminal

Screenshot:



```
larashtm@larasVM: ~/II2210_ x + v
Receiving objects: 100% (6/6), done.
larashtm@larasVM:~/II2210_Tucil-2$ nano main.py
larashtm@larasVM:~/II2210_Tucil-2$ nano model.py
larashtm@larasVM:~/II2210_Tucil-2$ nano requirements.txt
larashtm@larasVM:~/II2210_Tucil-2$ nano tester.py
larashtm@larasVM:~/II2210_Tucil-2$ nano Dockerfile
larashtm@larasVM:~/II2210_Tucil-2$ nano index.html
```

3.4. Melengkapi main.py

3.4.1. Endpoint: GET/

Perintah yang digunakan:

```
@app.get("/")
async def root():
    return FileResponse("index.html")
```


Langkah-langkah dan Penjelasan:

1. `@app.get("/")`

- Ini adalah **route decorator** dari FastAPI.
Artinya: saat ada **permintaan HTTP GET ke URL root ("/")**, maka fungsi **root()** di bawahnya akan dipanggil. Contoh: ketika membuka **http://localhost:8000** di browser, request GET akan dikirim ke path **/**, dan FastAPI akan jalankan fungsi **root()** ini.

2. `async def root():`

- Fungsi **root()** adalah **fungsi asynchronous** yang akan menangani request GET tadi.
- Penggunaan **async** memungkinkan FastAPI untuk menangani banyak request secara efisien (tanpa nunggu request selesai satu-satu).

3. `return FileResponse("index.html")`

- **FileResponse** akan mengirimkan **file index.html** sebagai **respon ke browser**. Jadi, browser akan menerima isi dari file **index.html**, lalu menampilkannya sebagai halaman web.

Screenshot:

```
# Users
users = {
    "sister": "ii2210_sister_rahasia",
    "laras": "ii2210_laras_sukaungu"
}

@app.get("/")
async def root():
    return FileResponse("index.html")
```

3.4.2. Endpoint: GET/motd

Perintah yang digunakan:

```
@app.get("/motd")
async def get_motd(session: SessionDep):
    motd = session.query(MOTD).order_by(func.random()).first()
```

```
if motd:
```

```
    return {"message": motd.motd}
```

```
return {"message": "No message found."}
```

Langkah-langkah dan Penjelasan:

1. @app.get("/motd")

- Ini adalah route FastAPI.
- Artinya, setiap kali ada request GET ke **/motd**, fungsi **get_motd()** akan dijalankan.

2. async def get_motd(session: SessionDep):

- Fungsi asynchronous (**async**) yang menerima satu parameter: **session**.
- **session** ini adalah **SQLAlchemy database session**, yang didapat dari **dependency injection** (SessionDep).

3. session.query(MOTD).order_by(func.random()).first()

- Ini artinya: ambil satu data acak dari tabel **MOTD**.
- **func.random()** mengacak urutan datanya.
- **.first()** ambil satu data pertama dari hasil yang diacak tadi.
- Hasilnya disimpan di variabel **motd**.

4. if motd:

- Kalau ada data yang ditemukan...

5. return {"message": motd.motd}

- ...balikin dictionary dengan key "message" dan isi dari kolom **motd.motd**.

6. return {"message": "No message found."}

- Kalau nggak ada data sama sekali, maka respon fallback ini yang dikembalikan.

Screenshot:

```
@app.get("/motd")
async def get_motd(session: SessionDep):
    motd = session.query(MOTD).order_by(func.random()).first()
    if motd:
        return {"message": motd.motd}
    return {"message": "No message found."}
```

3.4.3. Endpoint: POST/motd

Perintah yang digunakan:

```
@app.get("/motd")
async def post_motd(
    message: MOTDBase,
    session: SessionDep,
    credentials: Annotated[HTTPBasicCredentials,
    Depends(security)]
):
    current_password_bytes = credentials.password.encode("utf8")
    valid_username, valid_password = False, False

    try:
        if credentials.username in users:
            valid_username = True
            s =
base64.b32encode(users.get(credentials.username).encode("utf-8
")).decode("utf-8")
            totp = pyotp.TOTP(s=s, digest="SHA256", digits=8)
            valid_password =
secrets.compare_digest(current_password_bytes,
totp.now().encode("utf8"))
```

```

    if valid_username and valid_password:
        new_motd = MOTD(motd=message.motd,
creator=credentials.username)
        session.add(new_motd)
        session.commit()
        return {"message": "MOTD added successfully."}
    else:
        raise
HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Invalid userid or password.")
    else:
        raise
HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Invalid userid or password.")
except HTTPException as e:
    raise e

```

Langkah-langkah dan Penjelasan:

1. `async def post_motd(message: MOTDBase, session: SessionDep, credentials: Annotated[HTTPBasicCredentials, Depends(security)]):`

- Fungsi asynchronous (**`async`**) yang digunakan untuk menambahkan pesan baru (MOTD) ke database.
- Menerima tiga parameter:
 - **`message`**: input data MOTD dalam bentuk schema **`MOTDBase`**.
 - **`ession`**: koneksi database SQLAlchemy dari dependency **`SessionDep`**.
 - **`credentials`**: informasi login (username dan password)

dari Basic Authentication, diambil pakai
Depends(security).

2. current_password_bytes =

credentials.password.encode("utf8")

- Ubah password yang dikirim user jadi bentuk **bytes**, supaya bisa dibandingkan dengan password dari OTP.

3. valid_username, valid_password = False, False

- Inisialisasi dua variabel untuk menyimpan status validasi username dan OTP.

4. if credentials.username in users:

- Cek apakah username yang dikirim user ada di dictionary **users** (user yang valid).

5. s =

base64.b32encode(users.get(credentials.username).encode("utf-8")).decode("utf-8")

- Ambil secret dari user, encode ke base32. Ini **format yang diperlukan oleh TOTP**.

6. totp = pyotp.TOTP(s=s, digest="SHA256", digits=8)

- Membuat objek TOTP dengan:

- **s**: secret tadi,
- Algoritma hashing **SHA256**,
- Jumlah digit OTP = 8.

. valid_password =

secrets.compare_digest(current_password_bytes, totp.now().encode("utf8"))

7. if valid_username and valid_password:

- Cek apakah user valid dan OTP cocok.

8. . new_motd = MOTD(motd=message.motd, creator=credentials.username)

- Membuat instance data MOTD baru dengan isi pesan dan nama user.

9. session.add(new_motd)

- Tambahkan data MOTD ke dalam sesi database.

10.. session.commit()

- Simpan (commit) perubahan ke database.

11. return {"message": "MOTD berhasil ditambahkan."}

- Jika semua berhasil, balas ke user dengan pesan sukses.

12.. else: raise HTTPException(...)

- Kalau validasi gagal, balas error dengan status 401 (Unauthorized).

Screenshot:

```
@app.post("/motd")
async def post_motd(
    message: MOTDBase,
    session: SessionDep,
    credentials: Annotated[HTTPBasicCredentials, Depends(security)]
):
    current_password_bytes = credentials.password.encode("utf8")
    valid_username, valid_password = False, False

    try:
        if credentials.username in users:
            valid_username = True
            s = base64.b32encode(users.get(credentials.username).encode("utf-8")).decode("utf-8")
            totp = pyotp.TOTP(s=s, digest="SHA256", digits=8)
            valid_password = secrets.compare_digest(current_password_bytes, totp.now().encode("utf8"))

            if valid_username and valid_password:
                new_motd = MOTD(motd=message.motd, creator=credentials.username)
                session.add(new_motd)
                session.commit()
                return {"message": "MOTD added successfully."}
            else:
                raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid userid or password.")
        else:
            raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid userid or password.")
    except HTTPException as e:
        raise e
```

3.4.4. Autentikasi

Perintah yang digunakan:

```
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=17787, reload=True)
```

Langkah-langkah dan Penjelasan:

Autentikasi: boleh pake fungsi authenticate, boleh langsung manual (pilih hapus → valid).

- Endpoint: udah punya GET dan POST → aman banget.
- Struktur: ngerti apa yang tulis → ini penting banget.

- `if __name__ == "__main__":` → opsional

Screenshot:

```
if __name__ == "__main__":  
    import uvicorn  
    uvicorn.run("main:app", host="0.0.0.0", port=17787, reload=True)  
larashtm@larasVM:~/II2210_Tucil-2$
```

3.4.5. Kode backend Python pakai FastAPI

Perintah yang digunakan:

```
import secrets  
import base64  
import pyotp  
from fastapi import FastAPI, Depends, HTTPException, status  
from fastapi.templating import Jinja2Templates  
from fastapi.requests import Request  
from fastapi.security import HTTPBasic, HTTPBasicCredentials  
from sqlmodel import create_engine, Session, SQLModel  
from typing import Annotated  
from model import MOTD, MOTDBase  
from fastapi.responses import HTMLResponse, FileResponse  
from sqlalchemy.sql import func  
  
# SQLite Database  
sqlite_file_name = "motd.db"  
sqlite_url = f"sqlite:/// {sqlite_file_name}"  
connect_args = {"check_same_thread": False}
```

```
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLAlchemyModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

SessionDep = Annotated[Session, Depends(get_session)]
```

Langkah-langkah dan Penjelasan:

Kode tersebut adalah bagian dari backend dibuat menggunakan FastAPI dan SQLite. Di bagian ini, dilakukan setup awal seperti mendefinisikan nama file database (**motd.db**), membuat URL koneksi SQLite, dan menyiapkan argumen koneksi agar bisa diakses dari thread berbeda. Fungsi **create_db_and_tables()** digunakan untuk membuat tabel berdasarkan model yang sudah didefinisikan, dan fungsi **get_session()** berperan sebagai dependency injection untuk mengatur sesi database dalam setiap permintaan. Saat aplikasi dijalankan, instance FastAPI dibuat dan fungsi **create_db_and_tables()** langsung dipanggil agar tabel otomatis dibuat jika belum ada.

Screenshot:


```

larashtm@larasVM:~/II2210_Tucil-2$ cat main.py
import secrets
import base64
import pyotp
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.templating import Jinja2Templates
from fastapi.requests import Request
from fastapi.security import HTTPBasic, HTTPBasicCredentials
from sqlmodel import create_engine, Session, SQLModel
from typing import Annotated
from model import MOTD, MOTDBase
from fastapi.responses import HTMLResponse, FileResponse
from sqlalchemy.sql import func

# SQLite Database
sqlite_file_name = "motd.db"
sqlite_url = f"sqlite:/// {sqlite_file_name}"
connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, connect_args=connect_args)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

SessionDep = Annotated[Session, Depends(get_session)]

```

3.5. Membuat Dockerfile (Agar web Server Bisa Dijalankan di Port 17787)

Perintah yang digunakan:

#Menggunakan image Python sebagai base image

FROM python:3.10-slim

Set working directory di dalam container

WORKDIR /app

Salin requirements.txt dulu (biar bisa cache)

COPY requirements.txt .

Install dependencies

RUN pip install --no-cache-dir -r requirements.txt

Salin semua file project ke container

COPY . .

Expose port FastAPI kamu

EXPOSE 17787

Jalankan aplikasi pakai uvicorn

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "17787"]

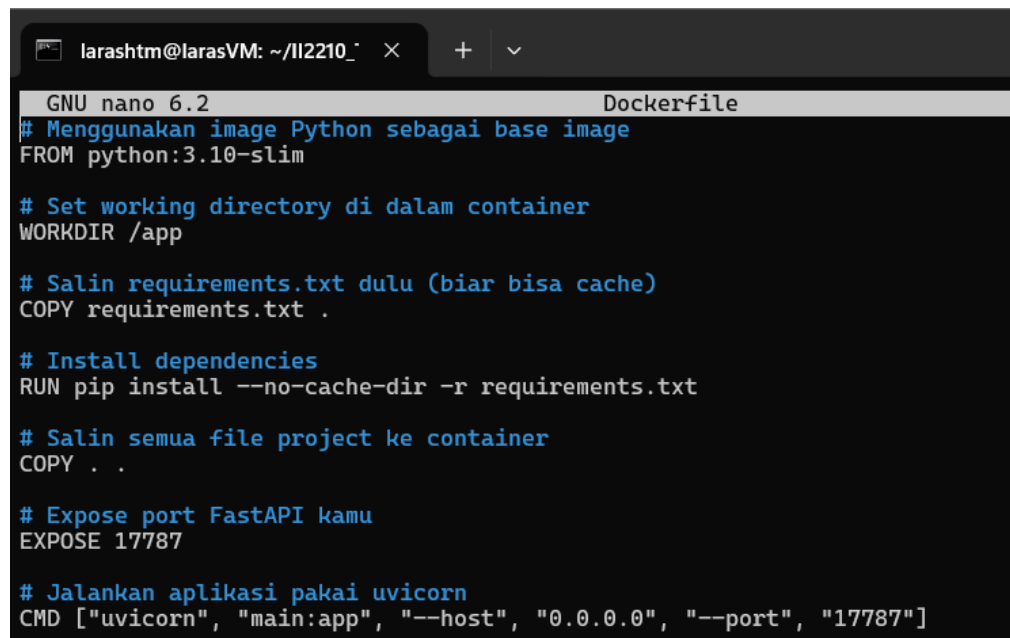
Langkah-langkah dan Penjelasan:

1. **FROM python:3.10-slim** menggunakan image Python versi 3.10 dengan varian slim sebagai base image. Slim dalam konteks ini berarti image ini ukurannya lebih kecil karena hanya menyertakan komponen minimal.
2. **WORKDIR /app** mensetting tempat penyimpanan di container ke /app. Semua perintah ini akan dijalankan dari folder.
3. **COPY requirements.txt** . menyalin requirements.txt dari folder lokal ke container di direktori /app. Hal ini dipisahkan supaya Docker bisa menggunakan cache.
4. **RUN pip install --no-cache-dir -r requirements.txt** menginstal seluruh Python dependencies yang terdaftar di **requirements.txt**.
--no-cache-dir dipakai agar cache pip tidak disimpan di dalam container (menghemat space).
5. **COPY . .** perintah ini digunakan untuk menyalin seluruh isi folder project ke

dalam container di direktori **/app**.

6. **EXPOSE 17787** perintah ini memberitahu kepada Docker bahwa container ini akan menggunakan port 17787 ini tidak membuka port, hanya memberi petunjuk.
 7. **CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "17787"]** perintah untuk menjalankan server FastAPI menggunakan Uvicorn.
- 3 aspek yang perlu diketahui secara bersama:
- **main:app** → Artinya ambil objek app dari file main.py
 - **--host, 0.0.0.0** → Biar bisa diakses dari luar container (bukan hanya localhost)
 - **--port, 17787** → Jalankan di port 17787

Screenshot:



The screenshot shows a terminal window with the title 'larashtm@larasVM: ~/112210_'. The editor is GNU nano 6.2, editing a file named 'Dockerfile'. The content of the Dockerfile is as follows:

```
# Menggunakan image Python sebagai base image
FROM python:3.10-slim

# Set working directory di dalam container
WORKDIR /app

# Salin requirements.txt dulu (biar bisa cache)
COPY requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Salin semua file project ke container
COPY . .

# Expose port FastAPI kamu
EXPOSE 17787

# Jalankan aplikasi pakai uvicorn
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "17787"]
```

3.6. Membuat index.html

Perintah yang digunakan:

```
<!DOCTYPE html>

<html lang="id">

<head>

  <meta charset="UTF-8" />

  <meta name="viewport" content="width=device-width,
initial-scale=1.0" />

  <title>Laras Ramalan Zodiak</title>

<style>

  body {

    font-family: 'Comic Sans MS', cursive, sans-serif;

    background-color: #f8f0ff;

    color: #4b006e;

    text-align: center;

    padding: 20px;

  }

  .card {

    background-color: #ffffff;

    border-radius: 15px;

    padding: 25px;

    max-width: 500px;
```

```
margin: auto;
```

```
box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
```

```
}
```

```
.question {
```

```
font-size: 20px;
```

```
margin-bottom: 15px;
```

```
}
```

```
.input-group {
```

```
margin-bottom: 20px;
```

```
}
```

```
button {
```

```
background-color: #9c4dcc;
```

```
color: white;
```

```
padding: 10px 20px;
```

```
border: none;
```

```
border-radius: 10px;
```

```
cursor: pointer;
```

```
font-size: 16px;
```

```
}
```

```
.sticker {
```

```
    font-size: 40px;

    margin-bottom: 10px;

}

</style>

</head>

<body>

<div class="card" id="step1">

    <div class="sticker">🌟</div>

    <div class="question">Namamu siapa nih?</div>

    <div class="input-group">

        <input type="text" id="nama" placeholder="Tulis namamu..." />

    </div>

    <button onclick="nextStep()">Next</button>

</div>

<div class="card" id="step2" style="display: none;">

    <div class="sticker">🎂</div>

    <div class="question">Berapa usiamu?</div>

    <div class="input-group">
```

```
<input type="number" id="usia" placeholder="Contoh: 19" />

</div>

<button onclick="nextStep()">Next</button>

</div>

<div class="card" id="step3" style="display: none;">

  <div class="sticker">♈♉♊♋♌♍</div>

  <div class="question">Zodiak kamu apa?</div>

  <div class="input-group">

    <select id="zodiak">

      <option value="">-- Pilih zodiakmu --</option>

      <option value="Aries">Aries</option>

      <option value="Taurus">Taurus</option>

      <option value="Gemini">Gemini</option>

      <option value="Cancer">Cancer</option>

      <option value="Leo">Leo</option>

      <option value="Virgo">Virgo</option>

      <option value="Libra">Libra</option>

      <option value="Scorpio">Scorpio</option>

      <option value="Sagittarius">Sagittarius</option>
```

```
<option value="Capricorn">Capricorn</option>

<option value="Aquarius">Aquarius</option>

<option value="Pisces">Pisces</option>

</select>

</div>

<button onclick="showResult()">Lihat Ramalan!</button>

</div>


<div class="card" id="result" style="display: none;">

  <div class="sticker"> 🧙 </div>

  <div class="question" id="ramalanOutput"></div>

  <button onclick="location.reload()">Coba Lagi</button>

</div>


<script>

  let step = 1;


  function nextStep() {

    document.getElementById("step" + step).style.display = "none";

    step++;
```



```
document.getElementById("step" + step).style.display = "block";

}

function showResult() {

    const nama = document.getElementById("nama").value || "Teman
Misterius";

    const usia = document.getElementById("usia").value || "Rahasia";

    const zodiak = document.getElementById("zodiak").value || "Alien";

    const ramalan = `Halo ${nama} si ${zodiak} berusia ${usia} tahun!

    Hari ini kamu kayak teh manis yang kelamaan di meja—manis tapi
    lelah.

    Jangan lupa istirahat ya. Semesta support kamu buat tetap jadi
    yang terbaik!`;

    document.getElementById("step3").style.display = "none";

    document.getElementById("ramalanOutput").innerText = ramalan;

    document.getElementById("result").style.display = "block";

}

</script>

</body>
```

```
</html>
```

Langkah-langkah dan Penjelasan:

1. **<!DOCTYPE html>** berarti dokumen ini adalah HTML5.
2. **<html lang="id">** bahasa utama halaman adalah Bahasa Indonesia (**id**).
3. **charset="UTF-8"** → Agar teks mendukung semua karakter (termasuk emoji 🌟).
4. **viewport** → Agar tampilannya responsif di HP.
5. **title** → Judul yang muncul di tab browser.
6. Background: **#f8f0ff** → warna background pastel
7. **.card** → box putih dengan rounded corner & bayangan.
8. Minta nama user → disimpan di **id="nama"**
Klik tombol akan lanjut ke step berikutnya lewat fungsi **nextStep()**.
9. Input usia user (angka), disimpan di **id="usia"** Awalnya **display: none;**, baru muncul setelah klik "Next".
10. fungsi **showResult()** untuk tampilkan ramalan.
11. Tombol **"Coba Lagi"** akan reload halaman.

Screenshot:


```
from sqlmodel import Field, SQLModel

from datetime import datetime

class MOTDBase(SQLModel):

    motd: str

class MOTD(MOTDBase, table=True):

    id: int | None = Field(default=None, primary_key=True)

    creator: str

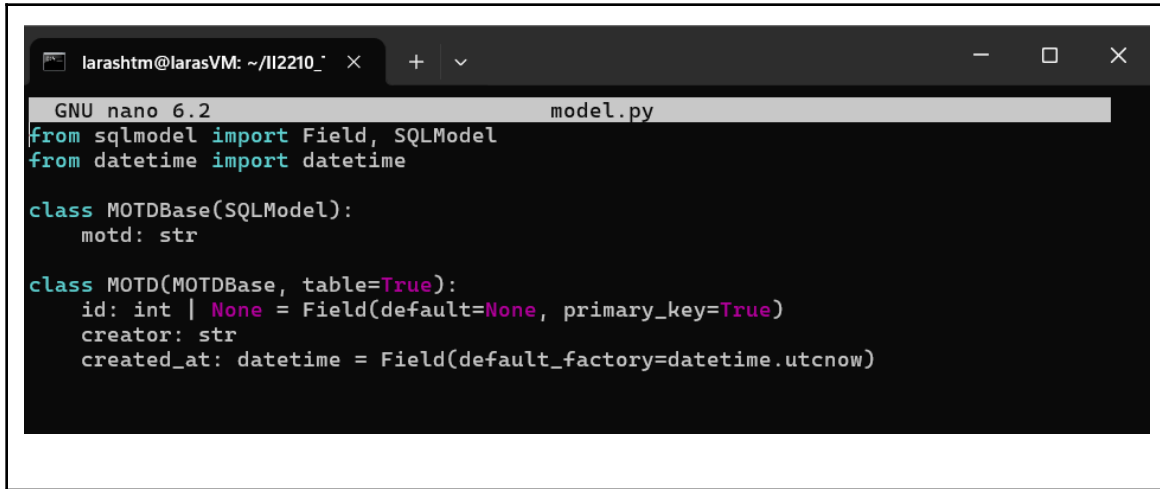
    created_at: datetime = Field(default_factory=datetime.utcnow)
```

Langkah-langkah dan Penjelasan:

Bagian **model.py** ini **tidak perlu di edit lagi** karena sudah lengkap dan sesuai standar untuk mendefinisikan model database pakai **SQLModel**. Penjelasannya **on point** kayak gini:

1. **MOTDBase** berisi field dasar (**motd**) biar reusable, biasanya untuk input/output schema.
2. **MOTD** adalah model tabel utama, pakai **table=True** untuknyatain ini akan jadi tabel di database.
3. **id** sebagai **primary key**, **creator** untuk menyimpan siapa yang untuk, dan **created_at** otomatis mengisi waktu sekarang pakai **default_factory=datetime.utcnow**.

Screenshot:



```
larashtm@larasVM: ~/l12210_ x + v
GNU nano 6.2 model.py
from sqlmodel import Field, SQLModel
from datetime import datetime

class MOTDBase(SQLModel):
    motd: str

class MOTD(MOTDBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    creator: str
    created_at: datetime = Field(default_factory=datetime.utcnow)
```

3.8. Membuat requirements.txt

Perintah yang digunakan:

fastapi==0.115.12

uvicorn==0.34.0

sqlmodel==0.0.24

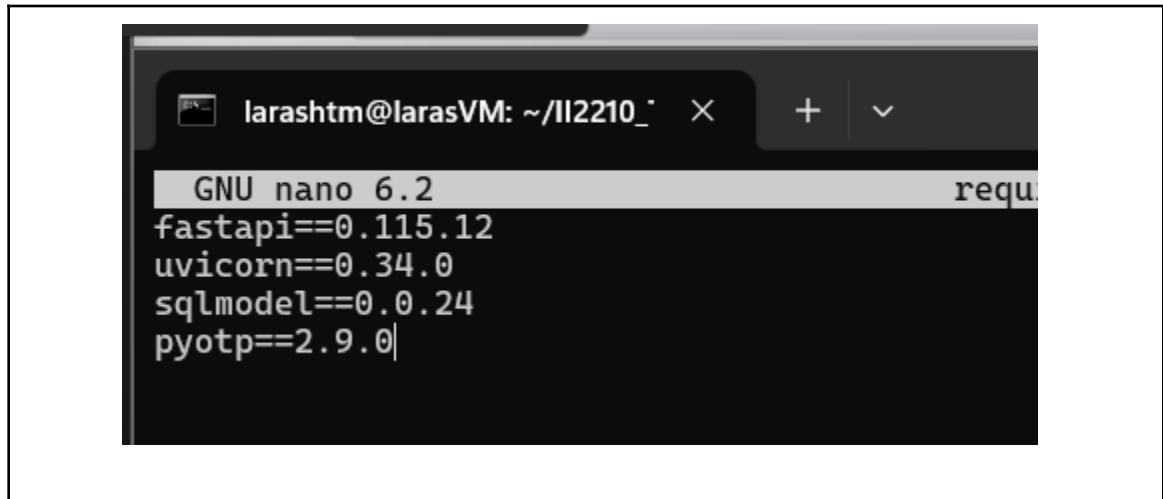
pyotp==2.9.0

Langkah-langkah dan Penjelasan:

1. **fastapi** – framework utama untuk bikin web API.
2. **uvicorn** – web server ringan dan cepat untuk menjalankan FastAPI.
3. **sqlmodel** – ORM (Object Relational Mapper) untuk mengatur database dengan simpel.

4. **pyotp** – mengatur autentikasi berbasis OTP (kayak Google Authenticator).

Screenshot:



3.9. Membuat tester.py

Perintah yang digunakan:

```
import requests

import pyotp

import time

import base64

import json


# ganti dengan userid dan shared secret yang ingin dicoba

userid = "laras"
```

```
shared_secret = "ii2210_laras_sukaungu"

# ganti dengan url server kalian

server_url = "http://20.2.164.44:17787//motd"

# ganti dengan motd yang diinginkan

motd = {"motd" : "Slow progres is still progress, jadi jangan nyerah yaa"}

s = base64.b32encode(shared_secret.encode("utf-8")).decode("utf-8")

totp = pyotp.TOTP(s=s,digest="SHA256",digits=8)

x = f"{userid}:" + totp.now()

a = "Basic " +
base64.b64encode(bytes(x,encoding="ascii")).decode("ascii")

resp = requests.get(url=server_url, headers={"Authorization" : a},
json=motd)

print(resp.content.decode("utf-8"))
```

Langkah-langkah dan Penjelasan:

1. Import Library

- **requests**: untuk mengirim HTTP request ke server
- **pyotp**: membuat kode OTP berbasis waktu (TOTP)
- **base64**: encoding string untuk otentikasi
- **json**: format data yang dikirim ke server

2. Setup User dan Server

- ``userid`` dan ``shared_secret`` digunakan untuk membuat token login
- ``server_url`` adalah alamat server yang ingin diakses

3. Membuat Pesan MOTD

- **MOTD** adalah pesan yang akan dikirim ke server dalam format JSON

4. Generate Token TOTP

- Shared secret diubah jadi format **base32** (dibutuhkan pyotp)
- Token TOTP dihasilkan dengan algoritma **SHA256** dan 8 digit

5. Membuat Authorization Header

- Username dan token digabung ``userid:TOTP``, lalu di-**encode base64**
- Hasilnya dijadikan header Authorization (tipe Basic)

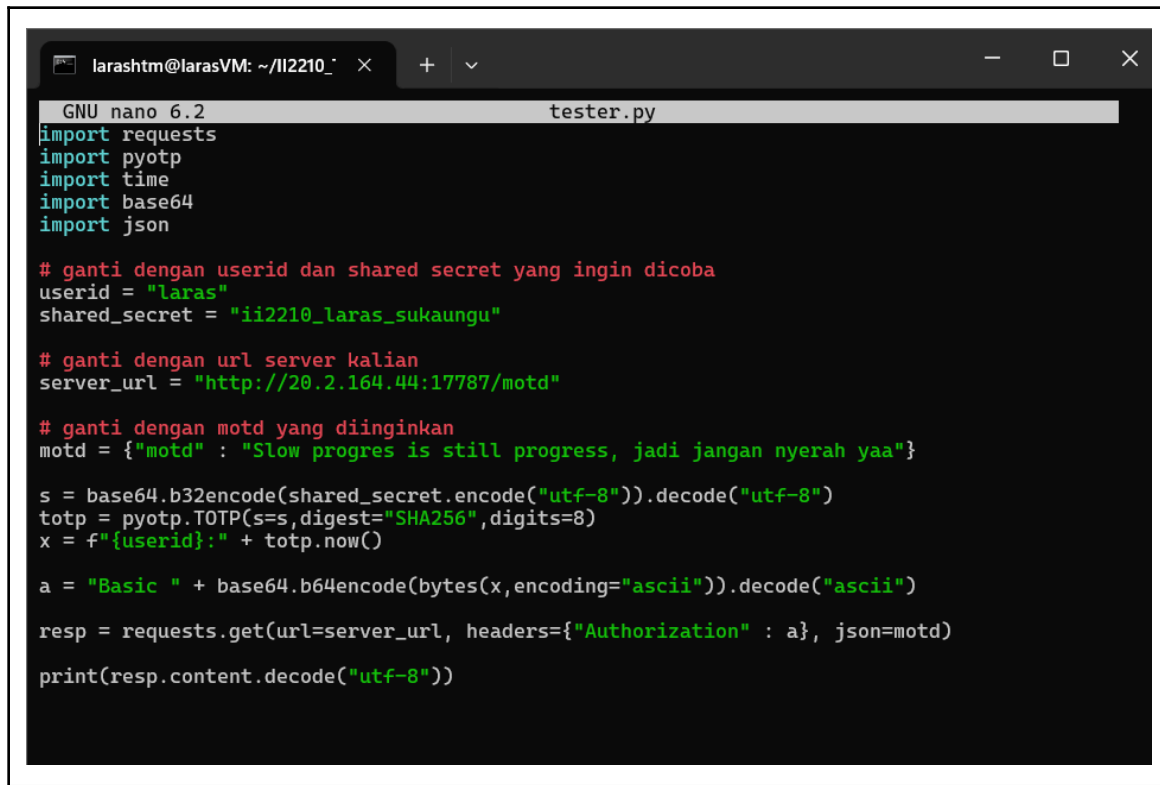
6. Kirim Permintaan ke Server

- Menggunakan metode **GET**
- Mengirim header Authorization dan JSON MOTD

7. Tampilkan Respon dari Server

- Respons server ditampilkan di terminal (berupa pesan sukses/gagal)

Screenshot:



```
larashtm@larasVM: ~/ii2210_ x + v
GNU nano 6.2 tester.py
import requests
import pyotp
import time
import base64
import json

# ganti dengan userid dan shared secret yang ingin dicoba
userid = "laras"
shared_secret = "ii2210_laras_sukaungu"

# ganti dengan url server kalian
server_url = "http://20.2.164.44:17787/motd"

# ganti dengan motd yang diinginkan
motd = {"motd" : "Slow progres is still progress, jadi jangan nyerah yaa"}

s = base64.b32encode(shared_secret.encode("utf-8")).decode("utf-8")
totp = pyotp.TOTP(s=digest="SHA256",digits=8)
x = f"{userid}:" + totp.now()

a = "Basic " + base64.b64encode(bytes(x,encoding="ascii")).decode("ascii")

resp = requests.get(url=server_url, headers={"Authorization" : a}, json=motd)

print(resp.content.decode("utf-8"))
```

3.10. Membuat docker-compose

Perintah yang digunakan:

nano docker-compose.yml

version: "3.8"

services:

web:

build: .

ports:

- "17787:17787"

volumes:

- **motd_data:/app/data**

volumes:

motd_data:

Langkah-langkah dan Penjelasan:

1. **version: "3.8"**: ini versi Docker Compose yang dipake. Versi 3.8 sudah cukup dan stabil dan mendukung
2. **services: web**: dibuat untuk mendeklarasi container-container yang ingin dijalankan yaitu service web.
3. **build**: build image-nya dari Dockerfile yang ada di folder ini (. = current folder).
4. - **"17787:17787"**: Nge-map port dari lokal ke dalam container.

port kiri 17787 = port di laptop (host), port kanan 17787 = port di dalam container tempat FastAPI-nya jalan. Jadi bisa akses FastAPI lewat <http://localhost:17787>

5. **volumes:**

- **motd_data:/app/data**: Ini bikin volume (penyimpanan data) yang bernama motd_data dan menyambung ke /app/data di dalam container. Ini berguna untuk simpan data biar nggak hilang saat container di-restart.

6. **command: uvicorn main:app --host 0.0.0.0 --port 17787 --reload**

Ini command untuk menjalankan FastAPI di dalam container pakai Uvicorn.

- **main:app** = artinya main.py dan objek app di dalamnya.

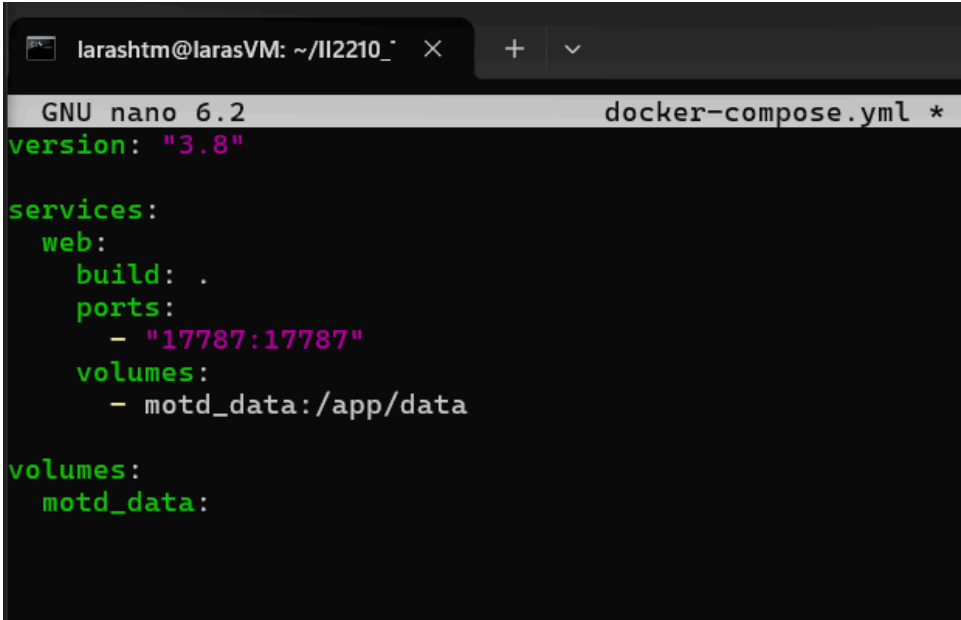
- **--host 0.0.0.0** = supaya bisa diakses dari luar container.
- **--port 17787** = port-nya.
- **--reload** = auto-restart kalau ada perubahan file (mirip hot-reload pas ngoding).

7. volumes:

motd_data:

Ini deklarasi volume yang tadi disebut. Docker akan bikin volume baru bernama motd_data.

Screenshot:



```
larashtm@larasVM: ~/l12210_ x + v
GNU nano 6.2 docker-compose.yml *
version: "3.8"

services:
  web:
    build: .
    ports:
      - "17787:17787"
    volumes:
      - motd_data:/app/data

volumes:
  motd_data:
```

3.11. Menjalankan Web Server menggunakan server menggunakan Docker Composer

Perintah yang digunakan:

```
sudo docker compose up --build -d
```

Langkah-langkah dan Penjelasan:

Perintah ini dipakai untuk membangun dan menjalankan container.

Screenshot:

```
larashtm@larasVM:~/II2210_Tucil-2$ sudo docker compose up --build
WARN[0000] /home/larashtm/II2210_Tucil-2/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please
nfusion
Compose can now delegate builds to bake for better performance.
To do so, set COMPOSE_BAKE=true.
[+] Building 2.1s (11/11) FINISHED                                docker:default
=> [web internal] load build definition from Dockerfile           0.0s
=> => transferring dockerfile: 506B                               0.0s
=> [web internal] load metadata for docker.io/library/python:3.10-slim 0.8s
=> [web internal] load .dockerignore                              0.0s
=> => transferring context: 2B                                     0.0s
=> [web 1/5] FROM docker.io/library/python:3.10-slim@sha256:06f6d69d 0.0s
=> [web internal] load build context                             0.0s
=> => transferring context: 12.24kB                                0.0s
=> CACHED [web 2/5] WORKDIR /app                                  0.0s
=> CACHED [web 3/5] COPY requirements.txt .                       0.0s
=> CACHED [web 4/5] RUN pip install --no-cache-dir -r requirements.t 0.0s
=> [web 5/5] COPY . .                                            0.1s
=> [web] exporting to image                                       0.8s
=> => exporting layers                                             0.8s
=> => writing image sha256:51f265f26255f25c8177bcb4359ea54e9442d5239 0.0s
=> => naming to docker.io/library/ii2210_tucil-2-web             0.0s
=> [web] resolving provenance for metadata file                  0.0s
[+] Running 2/2
✔web                               Built                               0.0s
✔Container ii2210_tucil-2-web-1    Recreated                         0.1s
Attaching to web-1
web-1 | INFO:      Started server process [1]
web-1 | INFO:      Waiting for application startup.
web-1 | INFO:      Application startup complete.
web-1 | INFO:      Uvicorn running on http://0.0.0.0:17787 (Press CTRL+C to quit)

larashtm@larasVM: ~/II2210_  x + v - □ ×
✔Network ii2210_tucil-2_default   Created                       0.1s
✔Container ii2210_tucil-2-web-1    Created                       0.1s
Attaching to web-1
web-1 | INFO:      Started server process [1]
web-1 | INFO:      Waiting for application startup.
web-1 | INFO:      Application startup complete.
web-1 | INFO:      Uvicorn running on http://0.0.0.0:17787 (Press CTRL+C to
quit)
Gracefully stopping... (press Ctrl+C again to force)
[+] Stopping 1/1
✔Container ii2210_tucil-2-web-1    Stopped                       0.7s
```

3.12. Konfigurasi akses port dan Uji Endpoint /motd (GET)

Langkah-langkah dan Penjelasan:

1. Masuk ke Azure Portal

Buka <https://portal.azure.com> dan login pakai akun Azure.

2. Buka Resource VM

- Masuk ke Virtual Machines.
- Klik nama VM yang sedang digunakan.
- 3. Akses Network Settings
 - Di menu sebelah kiri, pilih Networking.
 - Di bagian Inbound Port Rules, klik Add inbound port rule.
- 4. Tambah Rule Baru
 - Isi form sebagai berikut:
 - Source: Any
 - Source port ranges: *
 - Destination: Any
 - Destination port ranges: 17787
 - Protocol: TCP
 - Action: Allow
 - Priority: (misal) 300
 - Name: Allow-Port-17787
 - Description (opsional): Izinkan akses web server FastAPI

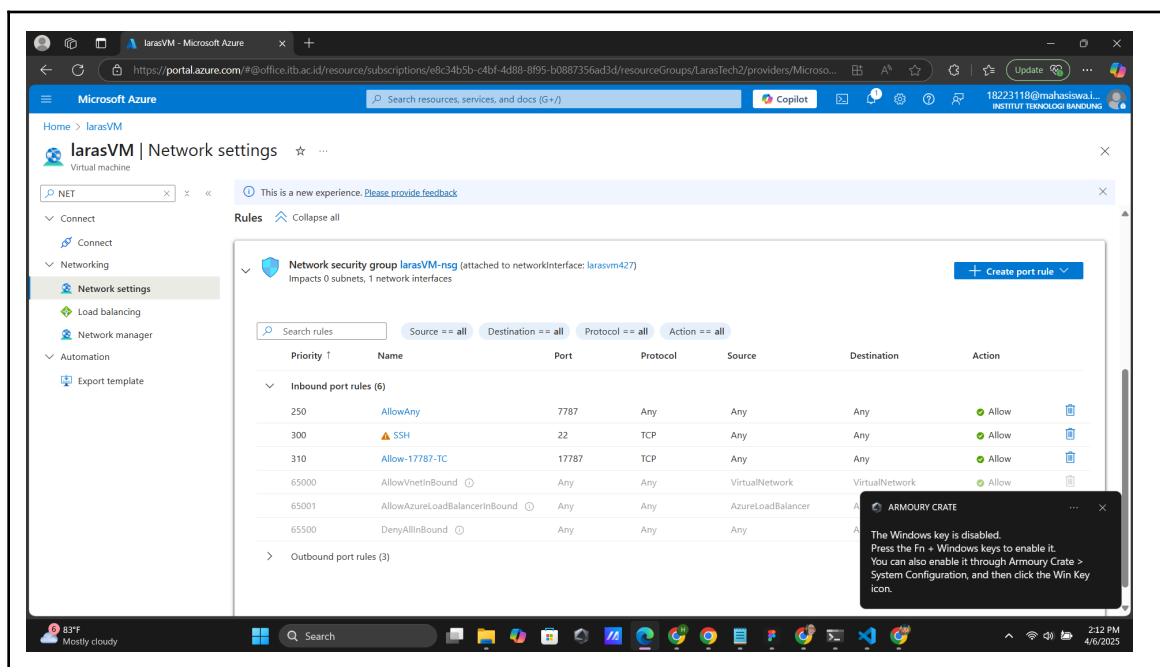
Klik Add untuk menyimpan rule.

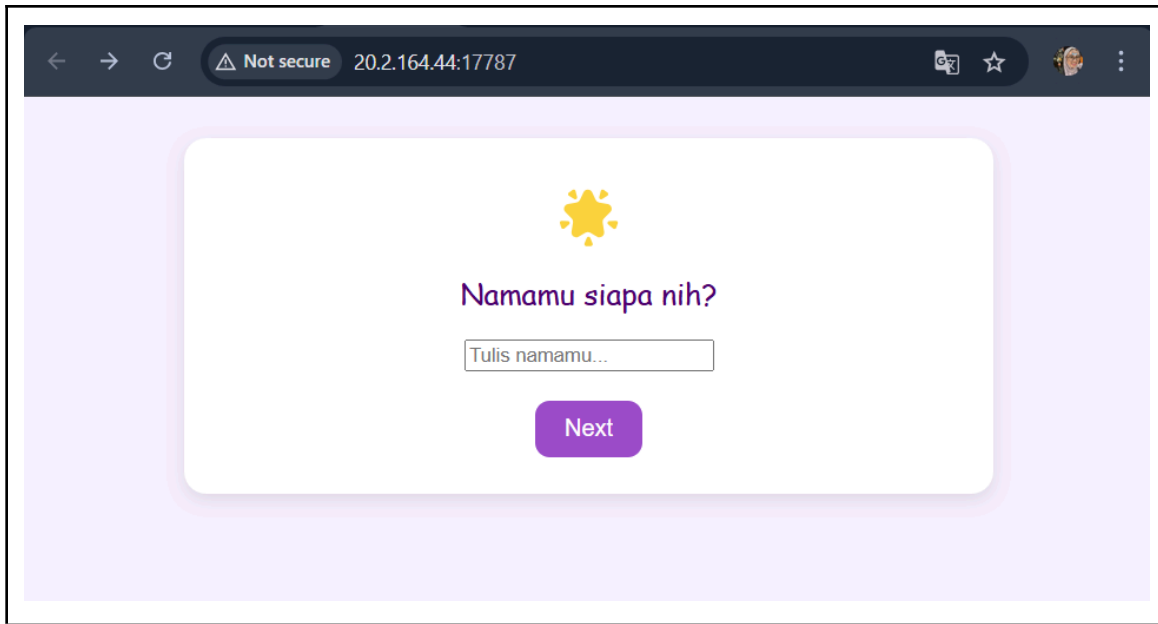
- 5. Verifikasi Akses
 - Setelah rule dibuat, buka browser dan akses:

<http://<public-ip-vm>:17787>

<http://20.2.164.44:17787/>

Screenshot:





3.13. Pengujian Endpoint /motd (GET)

Langkah-langkah dan Penjelasan:

Endpoint **/motd** digunakan untuk mengambil satu pesan acak dari database SQLite. Untuk menguji apakah endpoint ini berfungsi dengan benar, dilakukan pengujian menggunakan script **tester.py** yang telah dibuat sebelumnya. Script ini menggunakan library **requests** di Python untuk melakukan HTTP GET request ke endpoint yang dituju.

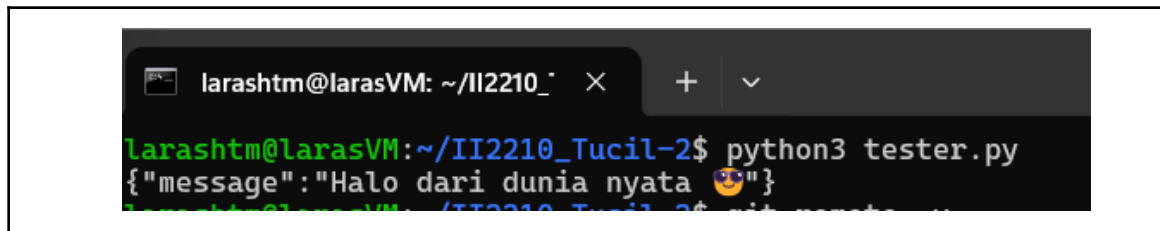
Berikut langkah-langkah pengujiannya:

1. Pastikan web server telah berjalan menggunakan perintah

```
docker-compose up.  
python3 tester.py
```

2. Output dari perintah tersebut akan menampilkan pesan acak dari database.
Contoh output:
{"message": "Halo dari dunia nyata 😎"}
3. Pengujian ini membuktikan bahwa endpoint **/motd** dapat diakses dengan benar dan berhasil mengambil data dari database.

Screenshot:

A screenshot of a terminal window with a dark background. The prompt is 'larashtm@larasVM: ~/II2210_'. The command 'python3 tester.py' has been executed, and the output is a JSON object: '{"message": "Halo dari dunia nyata 😊"}'.

```
larashtm@larasVM: ~/II2210_$ python3 tester.py
{"message": "Halo dari dunia nyata 😊"}
```

3.14. Pengujian Endpoint /motd (POST)

Langkah-langkah dan Penjelasan:

Endpoint ini digunakan untuk menambahkan pesan baru ke database. Endpoint ini dilindungi oleh autentikasi **HTTP Basic**, sehingga diperlukan username dan password saat mengaksesnya.

1. **Uvicorn main:app --host 0.0.0.0 --port 17787** => pengujian ini dilakukan untuk menyalakan port 17787. Jangan di Close biarkan port ini berjalan di 1 terminal.
2. Buka Cmd baru dan ketik python3 dan ketik satu satu seperti berikut:

python3

```
>>> import base64, pyotp
```

```
>>> s =
```

```
base64.b32encode("ii2210_laras_sukaungu".encode("utf-8")).decode("utf-8")
```

```
>>> totp = pyotp.TOTP(s=s, digest="SHA256", digits=8)
```

```
>>> totp.now() // perintah ini akan memunculkan kode token yang nantinya akan expired selama 30 detik
```

3. Buka CMD baru yang isinya untuk nge curl sebagai berikut:

Pengujian dapat dilakukan dengan perintah berikut:

```
curl -X POST "http://IP-KAMU:17787/motd" ^
-H "Content-Type: application/json" ^
```

```
-u laras:<OTP-TADI> ^
```

```
-d "{\"motd\": \"Halo dari dunia nyata 😎\"}"
```

Contoh:

```
curl -X POST "http://20.2.164.44:17787/motd" ^
```

```
-H "Content-Type: application/json" ^
```

```
-u laras:27012192 ^
```

```
-d "{\"motd\": \"Halo dari dunia nyata 😎\"}"
```

Screenshot:

```
larashtm@larasVM:~$ cd /II2210_Tucil-2
larashtm@larasVM:~/II2210_Tucil-2$ uvicorn main:app --host 0.0.0.0 --port 17787
INFO: Started server process [35484]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:17787 (Press CTRL+C to quit)
INFO: 114.10.144.63:35644 - "GET / HTTP/1.1" 200 OK
INFO: 114.10.144.63:35646 - "POST /motd HTTP/1.1" 401 Unauthorized
INFO: 114.10.144.63:35647 - "POST /motd HTTP/1.1" 200 OK
INFO: 114.10.149.63:50477 - "GET /motd HTTP/1.1" 200 OK
```

```
larashtm@larasVM:~/II2210_Tucil-2$ python3
Python 3.10.12 (main, Feb 4 2025, 14:57:36) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import base64, pyotp
>>> s = base64.b32encode("ii2210_laras_sukaungu".encode("utf-8")).decode("utf-8")
>>> totp = pyotp.TOTP(s=s, digest="SHA256", digits=8)
>>> totp.now()
'41194149'
```

```
C:\Users\laras\.ssh>curl -X POST "http://20.2.164.44:17787/motd" ^
More? -H "Content-Type: application/json" ^
More? -u laras:41194149 ^
More? -d "{\"motd\": \"Halo dari dunia nyata 😎\"}"
{"message":"MOTD added successfully."}
C:\Users\laras\.ssh>
```

4. Deployment dan Publikasi

4.1. Publikasi Kode ke GitHub (git remote)

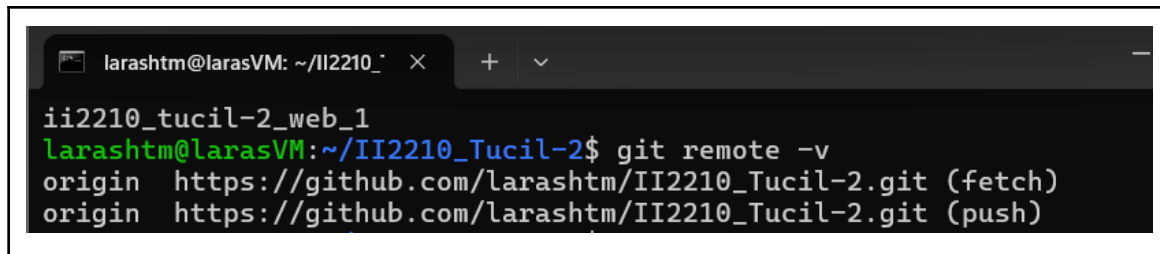
Perintah yang digunakan:

git remote -v

Langkah-langkah dan Penjelasan:

Perintah ini menampilkan URL dari repository yang terhubung, baik untuk fetch maupun push. Ini penting untuk memastikan bahwa semua perubahan kode bisa dipublikasikan ke GitHub. Setelah itu, saya juga menjalankan **git status** untuk melihat status file lokal, apakah ada perubahan yang belum di commit atau dipisah.

Screenshot:



```
larashtm@larasVM: ~/II2210_1 x + v
ii2210_tucil-2_web_1
larashtm@larasVM:~/II2210_Tucil-2$ git remote -v
origin https://github.com/larashtm/II2210_Tucil-2.git (fetch)
origin https://github.com/larashtm/II2210_Tucil-2.git (push)
```

4.2. Autentikasi SSH dan Cloning Repository

Langkah-langkah dan Penjelasan:

Sebelum menjalankan **git clone**, saya memastikan koneksi SSH sudah aktif. Berikut adalah langkah-langkah yang saya lakukan:

1. Mengecek isi folder **~/.ssh** menggunakan:

ls -l ~/.ssh

=> Menambahkan SSH key agar bisa digunakan untuk autentikasi GitHub:

ssh-add ~/.ssh/id_rsa

=> Setelah ini, muncul output seperti:

Identity added: /home/larashtm/.ssh/id_rsa

(larashtm@users.noreply.github.com)

Ini menandakan bahwa SSH key sudah berhasil ditambahkan.

2. Jika sebelumnya muncul error seperti
Permission denied (publickey)

fatal: Could not read from remote repository.

maka saya jalankan perintah:

eval "\$(ssh-agent -s)"

=> Ini akan mengaktifkan SSH agent agar bisa mengenali SSH key yang sudah ditambahkan.

3. Untuk memastikan koneksi ke GitHub sudah berhasil, saya jalankan:

ssh -T [git@github.com](https://github.com)

4. Output berhasil jika muncul pesan:

You've successfully authenticated, but GitHub does not provide shell access.

=> Setelah semua konfigurasi berhasil, saya melakukan **git clone** ke repository pribadi menggunakan:

git clone git@github.com:larashtm/II2210_Tucil-2.git

Screenshot:

```
larashtm@larasVM:~/II2210_Tucil-2/II2210_Tucil-2$ ls -l ~/.ssh
total 28
-rw----- 1 larashtm larashtm 664 Mar 26 15:55 authorized_keys
-rw----- 1 larashtm larashtm 399 Mar 26 06:29 id_ed25519
-rw-r--r-- 1 larashtm larashtm 92 Mar 26 06:29 id_ed25519.pub
-rw----- 1 larashtm larashtm 3481 Apr 6 18:20 id_rsa
-rw-r--r-- 1 larashtm larashtm 759 Apr 6 18:20 id_rsa.pub
-rw----- 1 larashtm larashtm 1120 Apr 5 23:49 known_hosts
-rw-r--r-- 1 larashtm larashtm 142 Mar 26 06:35 known_hosts.old
larashtm@larasVM:~/II2210_Tucil-2/II2210_Tucil-2$ cat ~/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQDAQDFQpnosS5lvqIKXF0CwVHTA700anIihC9+P9dC03wKcnLLJ4XLtDFA1Fsxkq70yu0aIyx2t/q/eiTV5HELHaF25edTvcG20+ix70edg8Wo/pbNrdTLmx
ttf6FGmjwPHpKIebkq3g7WC0nft+PktFkU8CteDaz9X/0mJr5LFDJ0qhtSK8.3kq1MsBOPGZL LwM186NufgC4CoNg8Rd3DKHydPae6Q4GSF1Kv02jF5NA6KuPH0Ny001wfc9J2VTUyNuhvpc88duJETpa50
b3+/JtOL3jpsY8FXUu533r08F904mktCS30nqUlaZ43+cgZ3/H09edKzghsPh2nb876nLb2RCKxw4P/fZjJawW4ZutddoTPdx09cnbfcZFL7e10ULcatyJuk8y+fyf880ctVWFR5HVoV2oaS3EAmb6sOC
80tYUL6CvcGiotUmsk6RQ0ueq0tQZhdFYn+1NhsSc4hi3D7GmaVUEga9n0h+KcZYVKctvahz7tRb66dnExeqUNMXQnz97DPdys8dtCVIHN3W2B68y6yCGx7wHhhyxUfsgg/xzjtnleJ6AS5VeanBYA
lhxcthaeMwMDDrouqr/gtxXakq3YtiykUESaVAMiLSXmk8PtyNv9tDvowFUmqBhl4pahHaqrMDPgYQOALfOYRwMo/4Ca8HdNw== larashtm@users.noreply.github.com
larashtm@larasVM:~/II2210_Tucil-2/II2210_Tucil-2$ git clone git@github.com:larashtm/II2210_Tucil-2.git
Cloning into 'II2210_Tucil-2'...
git@github.com: Permission denied (publickey).
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.
larashtm@larasVM:~/II2210_Tucil-2/II2210_Tucil-2$ eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_rsa
Agent pid 28574
Identity added: /home/larashtm/.ssh/id_rsa (larashtm@users.noreply.github.com)
larashtm@larasVM:~/II2210_Tucil-2/II2210_Tucil-2$ ssh -T git@github.com
-bash: syntax error near unexpected token `('
larashtm@larasVM:~/II2210_Tucil-2/II2210_Tucil-2$ ssh -T git@github.com
Hi larashtm/II2210_Tucil-2! You've successfully authenticated, but GitHub does not provide shell access.
larashtm@larasVM:~/II2210_Tucil-2/II2210_Tucil-2$ git status
```

4.3. Penggunaan git add, commit, dan push

Perintah:

```
git add Dockerfile
git commit -m "feat(docker): add Dockerfile for deployment"
git push origin main

git add docker-compose.yml
git commit -m "feat(docker): setup docker-compose configuration"
git push origin main

git add index.html
git commit -m "style(frontend): update homepage layout"
git push origin main

git add main.py
git commit -m "feat(api): implement FastAPI routes"
git push origin main

git add motd.html
git commit -m "feat(template): add MOTD display page"
git push origin main
git add recruitment.txt
git commit -m "docs: add recruitment info"
git push origin main
```

Langkah-langkah dan Penjelasan:

1. **git add <nama_file>**

- Fungsi: Memasukkan file ke *staging area*. Artinya, file tersebut sudah siap untuk di-commit ke repository.

git add Dockerfile

- Ini berarti **file Dockerfile** sudah disiapkan untuk dimasukkan ke commit selanjutnya.

2. **git commit -m "<pesan_commit>"**

- Fungsi: Merekam perubahan yang telah ditandai di *staging area* ke dalam repository lokal.
- Pesan commit berguna sebagai catatan tentang perubahan yang dilakukan, dan biasanya ditulis singkat, padat, dan jelas.

Contoh:

git commit -m "feat(docker): add Dockerfile for deployment"

- Ini menyimpan perubahan dari file **Dockerfile** dan memberi catatan bahwa file ini ditambahkan untuk keperluan deployment Docker.

3. **git push origin main**

- Fungsi: Mengirim commit dari repository lokal ke repository GitHub (remote).
- **origin** adalah nama default dari remote repository GitHub.
- **main** adalah nama branch utama.

Contoh:

git push origin main

Artinya commit terbaru yang ada di lokal dikirim ke branch **main** di GitHub.

Screenshot:

```
larashtm@larasVM:~/II2210_Tucil-2$ git add Dockerfile
larashtm@larasVM:~/II2210_Tucil-2$ git commit -m "feat(docker): add Dockerfile for containerization"
[main 95ee2a4] feat(docker): add Dockerfile for containerization
Committer: Ubuntu <larashtm@larasVM.mwbdfsny3fqe3bstt0oiohz0oa.hx.internal.cloudapp.net>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

1 file changed, 20 insertions(+)
larashtm@larasVM:~/II2210_Tucil-2$ git push --force
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 713 bytes | 713.00 KiB/s, done.
Total 3 (delta 0), reused 1 (delta 0), pack-reused 0
To github.com:larashtm/II2210_Tucil-2.git
+ 79dcf4d...95ee2a4 main -> main (forced update)
```

```
larashtm@larasVM: ~/II2210_
M      main.py
larashtm@larasVM:~/II2210_Tucil-2$ git add docker-compose.yml
larashtm@larasVM:~/II2210_Tucil-2$ git commit -m "feat(docker): add docker-compose.yml for service orchestration"
[main 90651a2] feat(docker): add docker-compose.yml for service orchestration
Committer: Ubuntu <larashtm@larasVM.mwbdfsny3fqe3bstt0oiohz0oa.hx.internal.cloudapp.net>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

1 file changed, 10 insertions(+)
create mode 100644 docker-compose.yml
larashtm@larasVM:~/II2210_Tucil-2$ git push --force
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 468 bytes | 468.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:larashtm/II2210_Tucil-2.git
   95ee2a4..90651a2  main -> main
```

```
larashtm@larasVM: ~/II2210_
larashtm@larasVM:~/II2210_Tucil-2$ git add index.html
larashtm@larasVM:~/II2210_Tucil-2$ git commit -m "feat(ui): add index.html as main page"
[main 8a23db4] feat(ui): add index.html as main page
Committer: Ubuntu <larashtm@larasVM.mwbdfsny3fqe3bstt0oiohz0oa.hx.internal.cloudapp.net>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

1 file changed, 48 insertions(+)
create mode 100644 index.html
larashtm@larasVM:~/II2210_Tucil-2$ git push --force
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 859 bytes | 859.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:larashtm/II2210_Tucil-2.git
   90651a2..8a23db4  main -> main
```

```
larashtm@larasVM: ~/II2210_ x + v
M      main.py
larashtm@larasVM:~/II2210_Tucil-2$ git add main.py
larashtm@larasVM:~/II2210_Tucil-2$ git commit -m "feat(IP): Implement FastAPI in main.py"
[main 88dd608] feat(IP): Implement FastAPI in main.py
Committer: Ubuntu <larashtm@larasVM.mwbdfsny3fqe3bstt0oiohz0oa.hx.internal.cloudapp.net>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

1 file changed, 148 insertions(+), 76 deletions(-)
rewrite main.py (60%)
larashtm@larasVM:~/II2210_Tucil-2$ git push --force
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 1.79 KiB | 1.79 MiB/s, done.
Total 3 (delta 1), reused 1 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:larashtm/II2210_Tucil-2.git
```

```
larashtm@larasVM:~/II2210_Tucil-2$ git commit -m "feat(db): add SQLAlchemy schema in model.py"
On branch main
Your branch is ahead of 'origin/main' by 2 commits.
(use "git push" to publish your local commits)

nothing to commit, working tree clean
larashtm@larasVM:~/II2210_Tucil-2$ git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Compressing objects: 100% (3/3), done.
```

```
larashtm@larasVM:~/II2210_Tucil-2$ git add motd.html
larashtm@larasVM:~/II2210_Tucil-2$ git commit -m "feat(motd.2): create motd.html for HTML template rendering"
[main df9b1d5] feat(motd.2): create motd.html for HTML template rendering
Committer: Ubuntu <larashtm@larasVM.mwbdfsny3fqe3bstt0oiohz0oa.hx.internal.cloudapp.net>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

1 file changed, 24 insertions(+)
create mode 100644 motd.html
larashtm@larasVM:~/II2210_Tucil-2$ git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 644 bytes | 644.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
```

```

larashtm@larasVM: ~/II2210_Tucil-2$ git add requirements.txt
larashtm@larasVM:~/II2210_Tucil-2$ git commit -m "chore(deps): add requirements.txt for dependencies management"
[main 1804713] chore(deps): add requirements.txt for dependencies management
Committer: Ubuntu <larashtm@larasVM.mwbdfsny3fqe3bstt0iohz0oa.hx.internal.cloudapp.net>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

1 file changed, 4 insertions(+)
create mode 100644 requirements.txt
larashtm@larasVM:~/II2210_Tucil-2$ git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 379 bytes | 379.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:larashtm/II2210_Tucil-2.git
   b56c902..1804713  main -> main

```

4.4. Deployment dengan Docker Compose

Perintah yang digunakan:

```
git pull ori main
```

Langkah-langkah dan Penjelasan:

Setelah perubahan dikirim ke GitHub, kita bisa menarik pembaruan dari branch utama (main) menggunakan:

git pull origin main: Sinkronisasi perubahan terbaru dari remote repository ke local. Ini memastikan tidak ada konflik antara versi lokal dan versi di GitHub.

Screenshot:

```

larashtm@larasVM:~/II2210_Tucil-2$ git pull origin main
From github.com:larashtm/II2210_Tucil-2
 * branch          main          -> FETCH_HEAD
Already up to date.

```

4.5. Mengedit dan Mengecek File docker-compose.yml

Perintah yang digunakan:

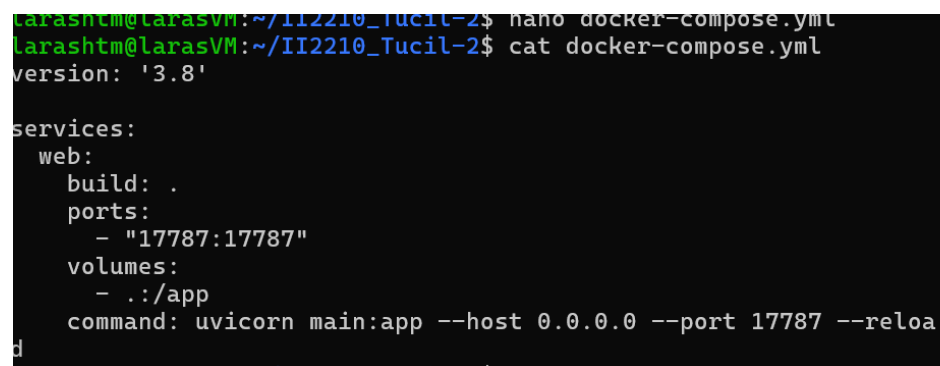
```
nano docker-compose.yml
cat docker-compose.yml
```

Langkah-langkah dan Penjelasan:

Sebelum menjalankan deployment, kita bisa mengedit file `docker-compose.yml` menggunakan:

- **nano docker-compose.yml**: Membuka file menggunakan teks editor di terminal.
- **cat docker-compose.yml**: Menampilkan isi file di terminal untuk verifikasi.

Screenshot:



```
larashtm@larasvm:~/II2210_Tucil-2$ nano docker-compose.yml
larashtm@larasvm:~/II2210_Tucil-2$ cat docker-compose.yml
version: '3.8'

services:
  web:
    build: .
    ports:
      - "17787:17787"
    volumes:
      - ./app
    command: uvicorn main:app --host 0.0.0.0 --port 17787 --reloa
```


4.6. Menjalankan Docker Compose

Perintah yang digunakan:

```
docker compose up -d --build
```

Langkah-langkah dan Penjelasan:

Setelah file **docker-compose.yml** siap, mulai proses deployment:

- **docker-compose up -d --build:** Menjalankan container di background (**-d**) dan membangun ulang image berdasarkan perubahan terakhir (**--build**).

Screenshot:

```
larashtm@larasVM:~/II2210_Tucil-2$ docker compose up -d --build
WARN[0000] /home/larashtm/II2210_Tucil-2/docker-compose.yml: the
attribute 'version' is obsolete, it will be ignored, please remove
it to avoid potential confusion
Compose can now delegate builds to bake for better performance.
To do so, set COMPOSE_BAKE=true.
[+] Building 3.7s (11/11) FINISHED                                docker:default
=> [web internal] load build definition from Dockerfile           0.0s
=> => transferring dockerfile: 526B                               0.0s
=> [web internal] load metadata for docker.io/library/pyt         1.7s
=> [web internal] load .dockerignore                             0.0s
=> => transferring context: 2B                                     0.0s
=> [web 1/5] FROM docker.io/library/python:3.10-slim@sha2        0.0s
=> [web internal] load build context                             0.2s
=> => transferring context: 5.93MB                                0.2s
=> CACHED [web 2/5] WORKDIR /app                                 0.0s
=> CACHED [web 3/5] COPY requirements.txt .                      0.0s
=> CACHED [web 4/5] RUN pip install --no-cache-dir -r req        0.0s
=> [web 5/5] COPY . .                                           0.4s
=> [web] exporting to image                                     1.1s
=> => exporting layers                                           1.1s
=> => writing image sha256:25340054c5603d94a9c29bclld2fc44      0.0s
=> => naming to docker.io/library/ii2210_tucil-2-web            0.0s
=> [web] resolving provenance for metadata file                 0.0s
[+] Running 2/2
✔web                               Built                                0.0s
✔Container ii2210_tucil-2-web-1   Started                             1.0s
```

4.7. Mengecek Container yang Sedang berjalan

Perintah yang digunakan:

```
ls
docker ps
```

Langkah-langkah dan Penjelasan:

1. Sebelum menjalankan perintah **docker ps**, saya juga menggunakan perintah **ls** untuk memastikan bahwa file dan direktori yang dibutuhkan, seperti **docker-compose.yml**, telah berada di dalam direktori kerja. Hal ini membantu memverifikasi struktur proyek sebelum memeriksa status container.
2. Perintah **docker-compose.yml** ini menampilkan daftar container yang sedang berjalan beserta port mappingnya.

Screenshot:

```
larashtm@larasVM:~/II2210_Tucil-2$ ls
Dockerfile          index_public.html  requirements.txt
II2210_Tucil-2      main.py            tester.py
docker-compose.yml  model.py
index.html          motd.html
larashtm@larasVM:~/II2210_Tucil-2$ |
```

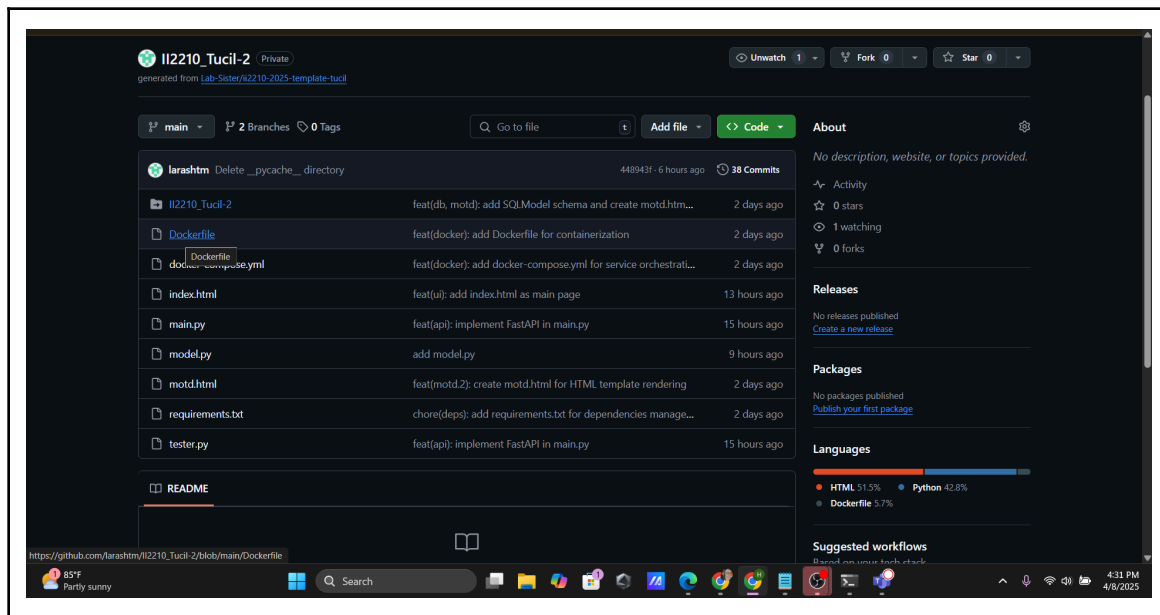
```
larashtm@larasVM:~/II2210_Tucil-2$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS
PORTS
942eb1e59478   ii2210_tucil-2-web  "uvicorn main:app --..."  About a minute ago  Up About
a minute      0.0.0.0:17787->17787/tcp, [::]:17787->17787/tcp  ii2210_tucil-2-web-1
```

4.8. Menampilkan Tampilkan Repo GitHub

Penjelasan:

Sebagai langkah akhir dari proses deployment dan publikasi, dilakukan pengecekan repository GitHub untuk memastikan bahwa seluruh file proyek telah berhasil di-*push*. Pada gambar berikut adalah tampilan dari folder proyek pada repository GitHub saya yang berisi file utama seperti **main.py**, **Dockerfile**, **docker-compose.yml**, **index.html**, dan file dokumentasi lainnya.

Screenshot:



5. Instalasi Docker dan Git di Virtual Machine

5.1. Menjalankan Container

Perintah yang digunakan:

```
docker ps
docker-compose up-d
```

Langkah-langkah dan Penjelasan:

Sebelum melakukan pengujian, dilakukan pengecekan container dengan perintah:

docker ps

Hasilnya akan menunjukkan kalau telah aktif dan berjalan di port 17787 dan ini memastikan bahwa server siap menerima request dari luar.

Screenshot:

```
larashtm@larasVM:~/ii2210_tucil-2$ docker ps
CONTAINER ID   IMAGE          COMMAND                  NAMES                  CREATED          STATUS
PORTS
836417b230cb   ii2210_tucil-2_web  "uvicorn main:app --..."  ii2210_tucil-2_web_1  18 minutes ago  Up 18 minutes
0.0.0.0:17787->17787/tcp, [::]:17787->17787/tcp
larashtm@larasVM:~/ii2210_tucil-2$ docker-compose up -d
ii2210_tucil-2_web_1 is up-to-date
```

5.2. Pengujian Endpoint //motd (GET)

Langkah-langkah dan Penjelasan:

Endpoint **/motd** digunakan untuk mengambil satu pesan acak dari database SQLite. Untuk menguji apakah endpoint ini berfungsi dengan benar, dilakukan pengujian menggunakan script **tester.py** yang telah dibuat sebelumnya. Script ini menggunakan library **requests** di Python untuk melakukan HTTP GET request ke endpoint yang dituju.

Berikut langkah-langkah pengujiannya:

1. Pastikan web server telah berjalan menggunakan perintah

docker-compose up.

python3 tester.py

2. Output dari perintah tersebut akan menampilkan pesan acak dari database. Contoh output:

{"message": "Halo dari dunia nyata 🤖"}

3. Pengujian ini membuktikan bahwa endpoint **/motd** dapat diakses dengan benar dan berhasil mengambil data dari database.

Screenshot:

```
larashtm@larasVM: ~/II2210_ x + v
larashtm@larasVM:~/II2210_Tucil-2$ python3 tester.py
{"message": "Halo dari dunia nyata 🤖"}
```

5.3. Pengujian Endpoint /motd (POST)

Endpoint ini digunakan untuk menambahkan pesan baru ke database. Endpoint ini dilindungi oleh autentikasi **HTTP Basic**, sehingga diperlukan username dan password saat mengaksesnya.

4. **Uvicorn main:app --host 0.0.0.0 --port 17787** => pengujian ini dilakukan untuk menyalakan port 17787. Jangan di Close biarkan port ini berjalan di 1 terminal.

5. Buka Cmd baru dan ketik python3 dan ketik satu satu seperti berikut:

python3

```
>>> import base64, pyotp
```

```
>>> s =
```

```
base64.b32encode("ii2210_laras_sukaungu".encode("utf-8")).decode("utf-8")
```

```
>>> totp = pyotp.TOTP(s=s, digest="SHA256", digits=8)
```

```
>>> totp.now() // perintah ini akan memunculkan kode token yang nantinya akan expired selama 30 detik
```

6. Buka CMD baru yang isinya untuk nge curl sebagai berikut:

Pengujian dapat dilakukan dengan perintah berikut:

```
curl -X POST "http://IP-KAMU:17787/motd" ^
```

```
-H "Content-Type: application/json" ^
```

```
-u laras:<OTP-TADI> ^
```

```
-d '{"motd": "Halo dari dunia nyata 🤖"}'
```

Contoh:

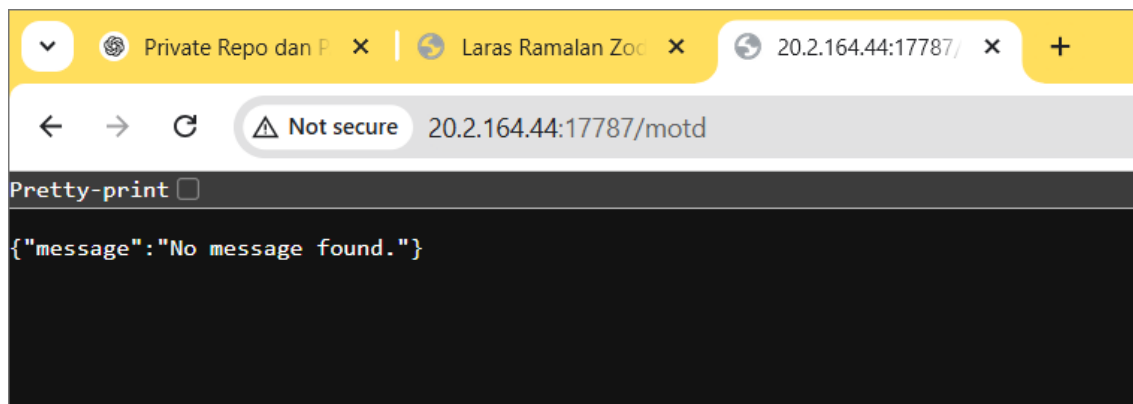
```
curl -X POST "http://20.2.164.44:17787/motd" ^  
-H "Content-Type: application/json" ^  
-u laras:27012192 ^  
-d "{\"motd\": \"Halo dari dunia nyata 😊\"}"
```

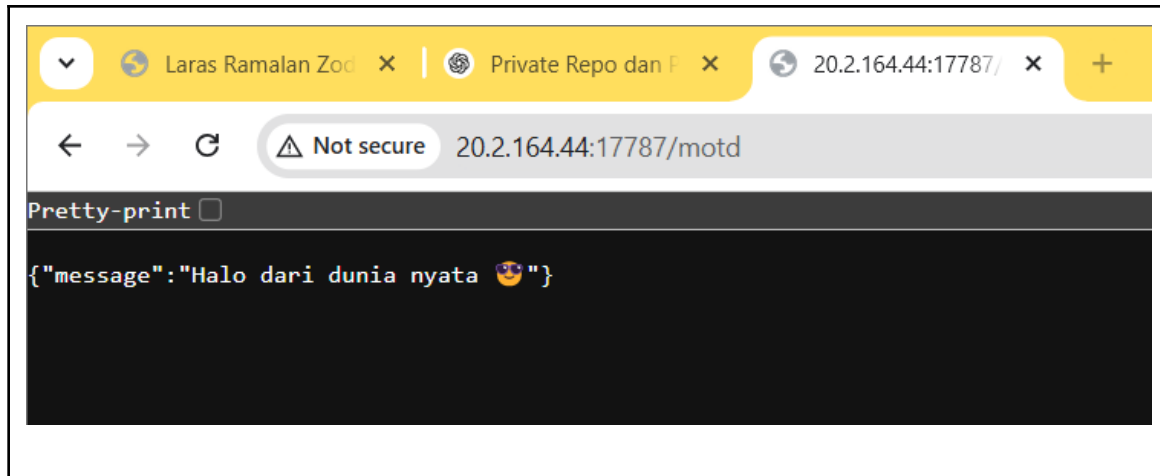
Screenshot:

```
larashtm@larasVM:~$ cd II2210_Tucil-2  
larashtm@larasVM:~/II2210_Tucil-2$ uvicorn main:app --host 0.0.0.0 --port 17787  
INFO: Started server process [35484]  
INFO: Waiting for application startup.  
INFO: Application startup complete.  
INFO: Uvicorn running on http://0.0.0.0:17787 (Press CTRL+C to quit)  
INFO: 114.10.144.63:35644 - "GET / HTTP/1.1" 200 OK  
INFO: 114.10.144.63:35646 - "POST /motd HTTP/1.1" 401 Unauthorized  
INFO: 114.10.144.63:35647 - "POST /motd HTTP/1.1" 200 OK  
INFO: 114.10.149.63:50477 - "GET /motd HTTP/1.1" 200 OK
```

```
larashtm@larasVM:~/II2210_Tucil-2$ python3  
Python 3.10.12 (main, Feb 4 2025, 14:57:36) [GCC 11.4.0] on linux  
Type "help", "copyright", "credits" or "license" for more information.  
>>> import base64, pyotp  
>>> s = base64.b32encode("ii2210_laras_sukaungu".encode("utf-8")).decode("utf-8")  
>>> totp = pyotp.TOTP(s=s, digest="SHA256", digits=8)  
>>> totp.now()  
'41194149'
```

```
C:\Users\laras\.ssh>curl -X POST "http://20.2.164.44:17787/motd" ^  
More? -H "Content-Type: application/json" ^  
More? -u laras:41194149 ^  
More? -d "{\"motd\": \"Halo dari dunia nyata 😊\"}"  
{"message": "MOTD added successfully."}  
C:\Users\laras\.ssh>
```





Perintah yang digunakan:

```
docker ps  
docker-compose up-d
```

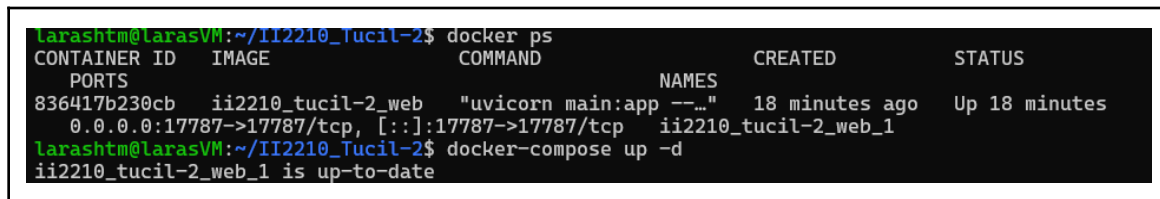
Langkah-langkah dan Penjelasan:

Sebelum melakukan pengujian, dilakukan pengecekan container dengan perintah:

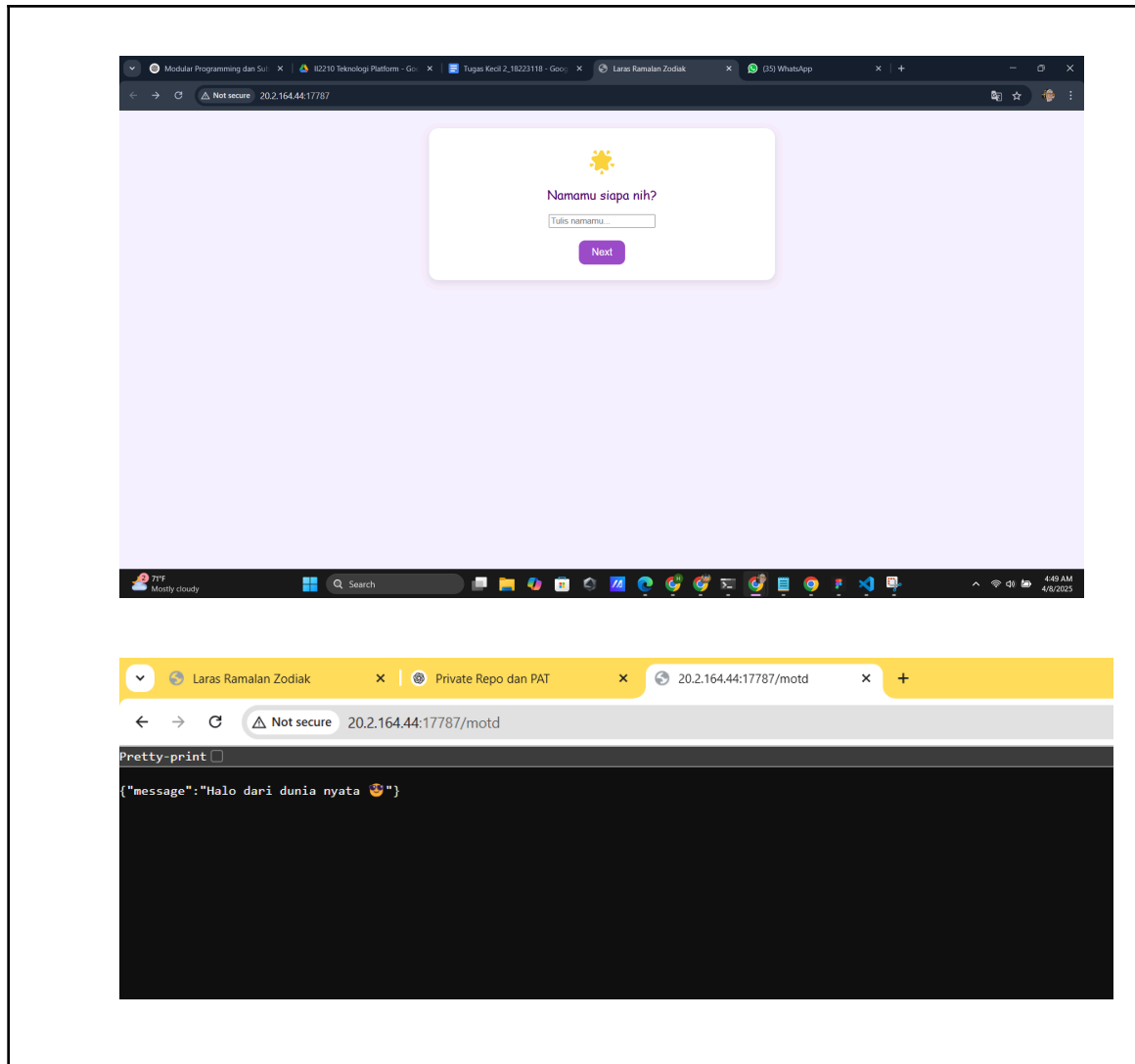
docker ps

Hasilnya akan menunjukkan kalau telah aktif dan berjalan di port 17787 dan ini memastikan bahwa server siap menerima request dari luar.

Screenshot:



Screenshot:



6. Bonus

6.1. Tampilan Endpoint GET /motd (Max +1)

Soal:

Endpoint GET /motd menampilkan message of the day menggunakan sebuah template HTML+CSS. Silahkan tuangkan kreativitas kalian ^_^ (Max +0.1, skala 0-4)

Langkah-langkah dan Penjelasan:


```
motd.html
<!-- motd.html -->
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Message of The Day</title>
  <style>
    body {
      font-family: 'Comic Sans MS', cursive, sans-serif;
      background-color: #f8f0ff;
      color: #4b006e;
      text-align: center;
      padding: 20px;
    }
    .card {
      background-color: #ffffff;
      border-radius: 15px;
      padding: 25px;
      max-width: 500px;
      margin: auto;
      box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
    }
    .sticker {
      font-size: 40px;
      margin-bottom: 10px;
    }
    .ramalan {
      font-size: 18px;
      white-space: pre-line;
    }
    button {
      background-color: #9c4dcc;
      color: white;
      padding: 10px 20px;
      border: none;
      border-radius: 10px;
      cursor: pointer;
      font-size: 16px;
      margin-top: 20px;
    }
  </style>
</head>
<body>
```

```

<div class="card">
  <div class="sticker">🔮</div>
  <div class="ramalan">
    {{ message }}
  </div>
  <form method="get">
    <button type="submit">Coba Lagi</button>
  </form>
</div>

</body>
</html>

```

Mengubah main.py

1. Tambahkan import:

Di bagian atas **main.py**, tambahkan:

```

from fastapi.templating import Jinja2Templates
from fastapi import Request

```

=> Agar dapat di render HTML pakai **Jinja2Templates**.

2. Tambahkan ini setelah **app = FastAPI(...)**:

```

templates = Jinja2Templates(directory="templates")

```

Pastikan file HTML disimpan(yang ramalan zodiak tadi) di folder bernama **templates/**, dan namanya misal **motd.html**.

3. Ganti (atau duplikat) endpoint **/motd** seperti ini:

```

@app.get("/motd/html", response_class=HTMLResponse)
async def get_motd_html(request: Request, session: SessionDep):
    motd = session.query(MOTD).order_by(func.random()).first()
    motd_message = motd.motd if motd else "Belum ada ramalan
MOTD 😊"
    return templates.TemplateResponse("motd.html", {"request":
request, "motd": motd_message})

```

4. Di dalam **motd.html**, edit bagian output supaya pakai Jinja2 syntax:

```
<!-- Ganti bagian showResult() atau bagian output -->
<div class="question" id="ramalanOutput">{{ motd }}</div>
```

Jadi HTML-nya terima **{{ motd }}** dari backend, bukan dari JS. Lalu setelah itu jalankan **uvicorn**:

```
uvicorn main:app --reload --host 0.0.0.0 --port 17787
```

lalu akses:

```
http://localhost:17787/motd/html
```

Perintah untuk menjalankan web server FastAPI:

```
uvicorn main:app --reload --host 0.0.0.0 --port 17787
```

- **uvicorn** → Ini adalah **ASGI server** yang dipakai untuk menjalankan FastAPI (semcam "mesin" yang mau distart).
- **main:app** → nama file Python nya yaitu **main.py**. Dengan **app**= objek FastAPI yang dibikin di **main.py**, yaitu **app = FastAPI()**.
- **--reload** → Otomatis restart server kalau ada perubahan di file (berguna saat ngoding).
- **--host 0.0.0.0** → Membuat server bisa diakses dari IP manapun (wajib dipakai kalau di VM/Cloud).
- **--port 17787** → Ini port custom, sesuai instruksi tugas.

Eksekusi Nya:

```

larashtm@larasVM:~/II2210_Tucil-2$ nano motd.html
larashtm@larasVM:~/II2210_Tucil-2$ nano main.py
larashtm@larasVM:~/II2210_Tucil-2$ ls
Dockerfile      docker-compose.yml  model.py      requirements.txt
II2210_Tucil-2  index.html          motd.db      tester.py
__pycache__     main.py             motd.html
larashtm@larasVM:~/II2210_Tucil-2$ mkdir templates
larashtm@larasVM:~/II2210_Tucil-2$ mv index.html templates/motd.html
larashtm@larasVM:~/II2210_Tucil-2$ ls templates/
motd.html
larashtm@larasVM:~/II2210_Tucil-2$

```

/your-project-folder

├── main.py

├── Dockerfile

├── requirements.txt

└── templates/

└── motd.html ← (tempelin file HTML yang udah dibikin)

Tambahin ini sebelum if

```
@app.get("/motd/html", response_class=HTMLResponse)
```

```
async def motd_html(request: Request, session: SessionDep):
```

```
    motd = session.query(MOTD).order_by(func.random()).first()
```

```
    return templates.TemplateResponse("motd.html", {
```

```
        "request": request,
```

```
        "message": motd.motd if motd else "No message found."
```

```
    })
```

Screenshot:


```
from fastapi.templating import Jinja2Templates
from fastapi.requests import Request
from fastapi.security import HTTPBasic, HTTPBasicCredentials
from sqlmodel import create_engine, Session, SQLModel
from typing import Annotated
from model import MOTD, MOTDBase
from fastapi.responses import HTMLResponse, FileResponse
from sqlalchemy.sql import func
```

```
# FastAPI
app = FastAPI(docs_url=None, redoc_url=None)
security = HTTPBasic()
templates = Jinja2Templates(directory="templates")
```

6.2. Otentikasi pada Endpoint POST /motd (Max +0,1)

Soal:

Jelaskan metode otentikasi yang digunakan pada *endpoint* POST /motd (Max +0.1, skala 0-4).

Penjelasan:

Endpoint **POST /motd** menggunakan metode otentikasi berbasis username dan password sederhana. Data **username** dan **password** dikirim sebagai bagian dari body JSON oleh client saat ingin menambahkan *message of the day* ke database.

Di sisi backend, aplikasi akan memverifikasi kredensial tersebut dengan mencocokkannya pada dictionary **allowed_users**, seperti berikut:

```
allowed_users = {

    "admin": "password123",

    "laras": "motdku"

}
```

Jika username dan password sesuai, maka server akan menambahkan pesan baru ke dalam database. Namun jika tidak cocok, maka server akan mengembalikan error **401 Unauthorized**. Dengan metode ini, hanya user yang memiliki kredensial valid yang bisa mengakses fitur **POST /motd**. Meskipun sederhana, metode ini sudah cukup untuk membatasi akses pada aplikasi skala kecil.

6.3. Cara Kerja SQLite (Max +0.05)

Soal:

Jelaskan cara kerja SQLite (Max +0.05, skala 0-4)

Langkah-langkah dan Penjelasan:

SQLite adalah sistem manajemen basis data relasional (RDBMS) yang bersifat ringan, serverless, dan self-contained, artinya tidak memerlukan proses server terpisah untuk dijalankan. Tidak seperti sistem basis data lainnya seperti MySQL atau PostgreSQL yang membutuhkan pengaturan server dan koneksi jaringan, SQLite bekerja langsung pada aplikasi yang menggunakannya dengan menyimpan seluruh data dalam satu file basis data berekstensi, db. **Cara Kerjanya:**

1. **Inisialisasi Database**

Saat aplikasi dijalankan, SQLite akan membuka (atau membuat jika belum ada) file **.db** sebagai tempat menyimpan data. Contohnya:

```
conn = sqlite3.connect("motd.db")
```

=> Di sinilah semua data, tabel, dan indeks disimpan secara terstruktur.

2. **Eksekusi SQL Query**

Aplikasi berinteraksi dengan SQLite menggunakan query SQL standar (misalnya **SELECT, INSERT, UPDATE, dll**). SQLite akan membaca query tersebut, mengeksekusinya, lalu menyimpan perubahan langsung ke file database.

3. **Transaksi dan Keamanan Data**

SQLite mendukung transaksi (*ACID-compliant*), sehingga semua operasi seperti insert/update akan dijalankan dalam transaksi yang aman, konsisten, dan dapat di-rollback jika gagal.

4. **Tanpa Koneksi Jaringan**

Karena tidak membutuhkan server, SQLite langsung diakses dari aplikasi. Ini membuatnya sangat cepat dan cocok untuk aplikasi kecil atau pengembangan lokal.

5. SQLite sering digunakan dalam:
 - Aplikasi desktop
 - Prototyping API
 - Aplikasi mobile
 - Sistem embedded (IoT, router, dll)

7. Catatan Tambahan (Disclaimer)

1. Alamat IP dari server sempat mengalami perubahan karena proses shutdown pada Virtual Machine yang menyebabkan restart IP publik. Oleh karena itu, jika terdapat perbedaan IP dalam dokumentasi tucil sebelumnya, hal tersebut terjadi akibat kondisi teknis dan bukan karena inkonsistensi pengerjaan.
2. “Seluruh spesifikasi wajib sudah bisa dijalankan dan diuji secara lokal. Fitur bonus telah dicoba dan berjalan di lokal, namun tidak berhasil di-*push* ke repository karena kendala teknis terkait konflik dan submodule. Dokumentasi dan cuplikan hasil telah disertakan sebagai bukti implementasi.”

8. Referensi

1. Docker
 - a. <https://docs.docker.com/engine/install/>
 - b. <https://docs.docker.com/get-started/docker-concepts/building-images/writing-a-dockerfile/>
2. Git basics
 - a. <https://www.w3schools.com/git/>
3. Github deploy key
 - a. <https://docs.github.com/en/authentication/connecting-to-github-with-ssh/managing-deploy-keys>
4. FastAPI
 - a. <https://fastapi.tiangolo.com/learn/>

- b. <https://fastapi.tiangolo.com/deployment/docker/>
- c. <https://sqlmodel.tiangolo.com/learn/>

9. Link Lainnya Mengenai Tugas

1. Link Web hasil: <http://20.2.164.44:17787/>
2. Link Git Hub: https://github.com/larashtm/II2210_Tucil-2